



UNIVERSITÄT ZU LÜBECK
INSTITUT FÜR INFORMATIONSSYSTEME

Centralized Task Planning and Dynamic Device Collaboration in LLM-based Multi-Agent Smart Home Systems

*Zentrale Task-Planung und dynamische Gerätekooperation in
LLM-basierten Multi- Agenten Smart-Home-Systemen*

Bachelorarbeit

verfasst am

Institute of Information Systems

im Rahmen des Studiengangs

Informatik

der Universität zu Lübeck

vorgelegt von

Zeyi Sun

ausgegeben und betreut von

Prof. Dr. rer. nat. habil. Sven Groppe

mit Unterstützung von

Jinghua Groppe

Lübeck, den 5. Januar 2026

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt, dass ich diese Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Zeyi Sun

Zusammenfassung

Aktuelle Smart-Home-Systeme stoßen oft an ihre Grenzen, da sie meist auf starren Regeln basieren, Sprache nicht wirklich verstehen und wenig flexibel sind. Um dieses Problem zu lösen, wurde in dieser Arbeit ein Multi-Agenten-System entwickelt, das auf lokal ausgeführten Large Language Models basiert.

Das System nutzt LangGraph als Framework und verbindet eine zentrale Planung mit der dezentralen Zusammenarbeit von Geräte-Agenten. Mithilfe der Reasoning-Fähigkeiten der LLMs kann das System natürliche Sprachbefehle analysieren und in konkrete, ausführbare Aktionen umwandeln. Um die Leistung objektiv zu messen, wurde ein eigener Benchmark für Smart-Home-Szenarien erstellt, der 60 Testfälle in den Kategorien einfach, mittel und komplex umfasst.

Die Ergebnisse zeigen eine Erfolgsquote von 80% bei einfachen Intentionen. Bei komplexen Intentionen lag die vollständige Erfolgsquote zwar bei 35%, jedoch konnte in weiteren 45% der Fälle ein Teilerfolg erzielt werden. Das zeigt: Auch wenn es bei langen Befehlsketten noch Optimierungsbedarf gibt, verfügt das System über die Fähigkeit zu Reasoning und Aufgabenplanung. Damit wird die Machbarkeit einer lokalen, LLM-basierten Architektur im Smart Home bestätigt.

Abstract

To address the limitations of current smart home systems—specifically their reliance on fixed rules, lack of semantic understanding, and insufficient flexibility, this thesis designs and implements a multi-agent smart home system based on locally deployed Large Language Models.

Using LangGraph as the core framework, the system establishes a mechanism that combines centralized planning with the dynamic collaboration of decentralized device agents. By leveraging the reasoning capabilities of LLMs, the system can analyze users’ natural language instructions into specific, executable device actions. To comprehensively evaluate the system’s performance, a smart home benchmark was constructed, consisting of 60 test cases across three difficulty levels: simple, moderate, and complex.

Experimental results show that the system achieves a success rate of 80% for simple intents. For complex intents, while the complete success rate is 35%, the partial success rate reaches 45%. These results indicate that although there is room for optimization in handling long-chain tasks, the system possesses the capability for reasoning and task planning. This validates the feasibility and potential of the locally deployed LLM-based multi-agent architecture in smart home scenarios.

Acknowledgements

I would like to express my sincere gratitude to Prof. Dr. Sven Groppe for his guidance and support throughout this thesis. His insightful feedback and encouragement were invaluable to this work. I am also grateful to Jinghua Groppe for her valuable assistance and support during my research. I am deeply grateful to my family, especially my mother, Jinghong Zhang, for their unwavering support and patience during my studies. Their encouragement has been a constant source of motivation. Finally, I would like to thank my friends for their companionship and moral support throughout this journey.

Contents

1	Introduction	1
1.1	Motivation and Background	1
1.2	Problem Statement	2
1.3	Research Objectives	2
1.4	Contributions	3
1.5	Thesis Organization	3
2	Basics and Related Work	4
2.1	Traditional Smart Home Architectures	4
2.2	Large Language Models as Autonomous Agents	5
2.3	Recent LLM-based Smart Home Architectures	6
2.4	LangGraph: A Framework for Stateful Multi-Agent Systems	7
2.5	Multi-Agent Coordination Theory	8
3	System Design and Implementation	10
3.1	Overall Architecture and Components	10
3.2	System Workflow	11
3.3	Design Rationale	13
3.4	Implementation Framework	14
3.5	Centralized Task Planning	17
3.6	Dynamic Device Collaboration Implementation	21
3.7	Complete Example: Bedtime Routine	25
4	Benchmark	30
4.1	Introduction	30
4.2	Design Methodology	30
4.3	Dataset Overview	32
4.4	Evaluation Criteria	33
5	Evaluation	35
5.1	Experimental Setup	35
5.2	Overall Performance	35
5.3	Performance Analysis by Case Examples	38
5.4	Discussion	42

6	Conclusion	45
6.1	Summary	45
6.2	Contributions	45
6.3	Future Work	46
	Bibliography	47
A	Source Code Repository	49
B	Test Cases and Results	50
B.1	Simple Test Cases	50
B.2	Moderate Test Cases	52
B.3	Complex Test Cases	54

1

Introduction

1.1 Motivation and Background

As smart products become widespread, people increasingly expect their lives to become more convenient, a trend evident in the rapid expansion of the smart home market [6].

A smart home, as an information technology-enabled system, integrates multiple smart objects and household devices. These components are capable of mutual communication and information exchange, with the core objective of providing services to users and facilitating convenient remote-control functionalities.[15, 5, 4]

Currently, traditional smart home systems (e.g., Alexa, Google Home, Mi Home)[20] typically rely on preset automation rules and lack the contextual understanding of user requests. Users are no longer satisfied with simple device control. Instead, they expect systems to show a higher level of intelligence, stronger responsiveness, and the ability to coordinate multiple devices to create a comfortable living environment.

To achieve these goals, modern smart home systems increasingly rely on Internet of Things (IoT) [7]technology to connect various devices and data exchange. However, traditional IoT architectures often face significant limitations in handling complex scenarios and understanding contextual information.

In recent years, the rapid development of Large Language Models (LLMs) has provided new opportunities to overcome these limitations. LLMs demonstrate powerful capabilities in natural language understanding and generation[13], allowing them to parse complex or even ambiguous user commands. Furthermore, they possess powerful common-sense reasoning and contextual understanding abilities, enabling them to infer user intent, perceive environmental states, and generate contextually appropriate task plans or control decisions. Therefore, Large Language Models are expected to compensate for the shortcomings of traditional rule-based systems in terms of flexibility and semantic understanding, creating the conditions for building smart home systems that are more intelligent, adaptive, and user-centric.

As research progresses, an increasing number of studies are exploring the introduction of LLMs as Agents into smart home systems, constructing Multi-Agent Systems (MAS)[8] where multiple agents collaborate. In such systems, typically, a high-level planning agent is responsible for task decomposition and decision-making, while multiple device agents execute their respective sub-tasks. However, achieving efficient task planning and inter-

agent collaboration within these MAS has become a new core challenge.

1.2 Problem Statement

In the field of Large Language Model based multi-Agent systems for smart home, the SAGE (Smart Home Agent with Grounded Execution)[16] architecture represents a significant advancement. SAGE orchestrates the use of tools in a sequential decision-making process and integrates a collection of novel components designed to tackle key challenges in smart home automation, including personalization, flexible device interaction, device monitoring, and device disambiguation, and it achieves a high success rate of 75% on complex tasks.[16]

However, the SAGE still faces practical deployment challenges. First, SAGE’s strong performance relies primarily on the most powerful and expensive proprietary LLMs (e.g., GPT-4) , which results in high querying costs. Second, SAGE functions as a centralized LLM orchestrator that executes device operations through unified tool calls, rather than enabling decentralized collaboration among device agents.

Given these limitations, this study aims to explore an architecture that combines centralized task planning with dynamic decentralized agent collaboration, which we call a hybrid architecture. In this design, a central planner decomposes user requests into high-level tasks, while individual device agents execute these tasks on their own and collaborate directly when needed. This approach addresses both issues with SAGE: using open-source LLMs deployed locally reduces costs, and allowing agents to collaborate autonomously reduces centralized control.

Specifically, this research addresses two core challenges:

1. **Task Generation:** How to design a central planner to effectively decompose complex, natural language user requests into a sequence of high-level, sequentially executable tasks. The key here is that the high-level task partitioning must remain abstract enough to avoid over-predicting the specific collaboration details between device agents.
2. **Dynamic Collaboration:** Determining whether a decentralized, peer-to-peer, and single round collaboration mechanism is sufficient to enable LLM-based device agents to autonomously determine (based on their task requirements) when collaboration is necessary and to stably resolve common task dependencies.

1.3 Research Objectives

This study aims to design, implement, and evaluate a hybrid multi-agent smart home architecture that combines centralized task planning with dynamic, decentralized collaboration among LLM-based device agents. Specific objectives include:

1. Design a centralized Task Planner that can effectively interpret complex user inputs and decompose them into a sequence of high-level tasks without embedding assumptions about device-to-device collaboration.
2. Develop a collaboration mechanism where device agents autonomously decide when and how to collaborate based on task requirements.

3. Construct a hybrid multi-agent system architecture. The evaluation focuses on whether this architecture can maintain a reasonable task success rate while accommodating the performance variability of open-source LLMs.
4. Develop a benchmark for evaluating smart home systems in handling natural language requests, with test cases of varying complexity and a systematic evaluation framework.
5. Test and evaluate the system using natural language user input of varying complexity levels.

1.4 Contributions

The contributions of this thesis are:

1. **System Design with Hybrid Coordination:** Design and implementation of a multi-agent smart home system combining centralized task planning with decentralized device collaboration, demonstrating effective integration of these coordination approaches using LLMs.
2. **Dynamic Collaboration Mechanism:** Implementation of a state-driven collaboration mechanism where agents autonomously determine collaboration needs through LLM-based reasoning, rather than following pre-defined rules.
3. **Functional Proof-of-Concept:** A working system with 8 device agents implemented using LangGraph, demonstrating the feasibility of the proposed design approach.
4. **Domain-Specific Benchmark:** Addressed the lack of domain-specific evaluation tools by creating a dataset of 60 test cases. The benchmark is designed to rigorously test the system’s performance across varying levels of natural language input complexity

1.5 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 reviews relevant literature, covering traditional smart home architectures, the evolution of LLMs into autonomous agents, recent LLM-based smart home systems, and the LangGraph framework for multi-agent coordination.

Chapter 3 presents the design and implementation of the proposed hybrid multi-agent system, including the overall architecture, system workflow, and implementation details using LangGraph.

Chapter 4 describes the testing methodology and benchmark construction, including 60 test cases categorized by complexity (simple, moderate, complex) and the three-dimensional evaluation framework.

Chapter 5 analyzes the experimental results, providing quantitative performance assessment across complexity levels and qualitative analysis of case studies to discuss system strengths and limitations.

Chapter 6 summarizes the thesis contributions, acknowledges current limitations, and proposes future research directions.

2

Basics and Related Work

This chapter reviews existing work related to this thesis. We will discuss the limitations of traditional smart home systems, the LLM-based solutions, and recent multi-agent architectures that motivate our approach.

2.1 Traditional Smart Home Architectures

Traditional smart home assistants like Amazon Alexa, Google Assistant, and Samsung Bixby are fundamentally rule-based systems. Their “intelligence” is fundamentally defined by connectivity and automation: the ability to connect devices to a cloud hub and execute pre-defined routines [11]. They work well for executing simple, direct commands like “turn on the kitchen light”, but they struggle with more complex or implicit requests.

Existing automation architectures often fall under the category of “semi-automated” systems. They rely heavily on users to manually configure rules via interfaces, which not only places a significant configuration burden on the user but also results in a lack of adaptability. Consequently, these systems are limited to mechanically executing pre-defined conditions, rather than proactively inferring dynamic user needs [9].

The main problem is that these systems cannot understand vague or under-specified commands. For example, if a user says “help me relax”, the system does not know what actions to take because it lacks the ability to infer user intent and plan accordingly [10]. This limitation has led researchers to explore more intelligent solutions using large language models.

Aspect	Traditional Systems	LLM-Based Systems
Command understanding	Keyword matching	Natural language understanding
Intelligence	Pre-defined IF-THEN rules	Autonomous reasoning & planning
Adaptability	Manual configuration required	Dynamic adaptation
User intent	Cannot infer from vague commands	Infers intent from context
Example command	“Turn on the kitchen light”	“Help me relax”

Table 2.1: Comparison of Smart Home System

2.2 Large Language Models as Autonomous Agents

Before discussing the specific role of LLMs, we first introduce the broader concept of Agentic AI. Agentic AI refers to autonomous systems capable of pursuing complex goals with minimal human supervision. Unlike traditional AI models that rely on rigid instructions for specific tasks, Agentic AI focuses on adaptability and independent decision-making, allowing it to function effectively in dynamic environments. This approach shifts the utility of AI from passive execution to active problem-solving and strategic planning. To achieve this autonomy, these systems typically incorporate reasoning mechanisms to prioritize actions and tool integration to interact with external systems. [1]

While Agentic AI provides the conceptual basis for this autonomy, Large Language Models (LLMs) have emerged as a practical foundation to realize these capabilities. The limitations of rule-based systems, coupled with the rapid progress of Large Language Models, have motivated researchers to adopt LLMs as the new “brain” for smart home agents. This shift represents a transition from executing pre-defined heuristic functions to acting based on comprehensive internal world knowledge [18], allowing for autonomously generating plans based on user goals. LLMs introduce two critical capabilities that are absent in traditional systems:

1. **Reasoning and Planning:** Through techniques such as chain-of-thought prompting and frameworks like ReAct [21], LLMs can decompose abstract natural language goals into simpler, executable subtasks [3]. This enables interpretation of vague user intents and generation of multi-step plans grounded in common-sense reasoning.
2. **Tool Use:** An LLM’s reasoning is not confined to text generation, it can interact with the external environment. By “tool use” or “function calling,” an LLM can decide *which* device to control and *how* to control it by generating the necessary API calls. The SAGE [16] system is a prime example of an LLM augmented with a “tool set” to retrieve information and manipulate device states.

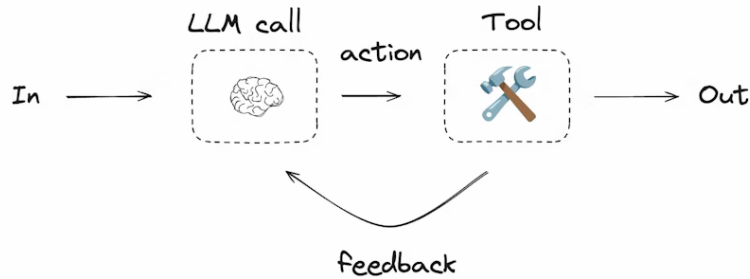


Figure 2.2: Basic structure of an LLM-based autonomous agent. The agent operates in a continuous feedback loop: receiving input, invoking the LLM for reasoning, executing tool actions based on LLM decisions, and incorporating tool feedback to inform subsequent steps. This architecture enables agents to autonomously plan and execute multi-step tasks.

Figure 2.2 (Source: LangChain Documentation¹).

¹<https://docs.langchain.com/oss/python/langgraph/workflows-agents>

However, while a single, powerful LLM agent can plan and act, applying it to a complex environment like a smart home (with dozens of devices) introduces a new challenge. A single agent responsible for planning, reasoning, and executing every action for every device becomes inefficient and difficult to scale. This limitation naturally leads to the adoption of **Multi-Agent Systems (MAS)**, where specialized agents (e.g., a “lighting agent”, “thermostat agent”) manage their own domains.

2.3 Recent LLM-based Smart Home Architectures

As LLM agents emerged as a viable solution, the first and most intuitive coordination pattern to appear was centralized orchestration. In this model, a single planner or orchestrator agent is responsible for decomposing the user’s goal and dictating the plan for all other components.

An example of this approach is Sasha [10]. The Sasha system processes user commands through a manually defined, fixed pipeline consisting of five stages: Clarifying, Filtering, Planning, Execution, and Feedback [10]. While this was an improvement over rule-based systems, its core limitation, as later identified by the SAGE paper [16], is this rigid, pre-defined sequence, which restricts its flexibility in handling dynamic or unexpected scenarios.

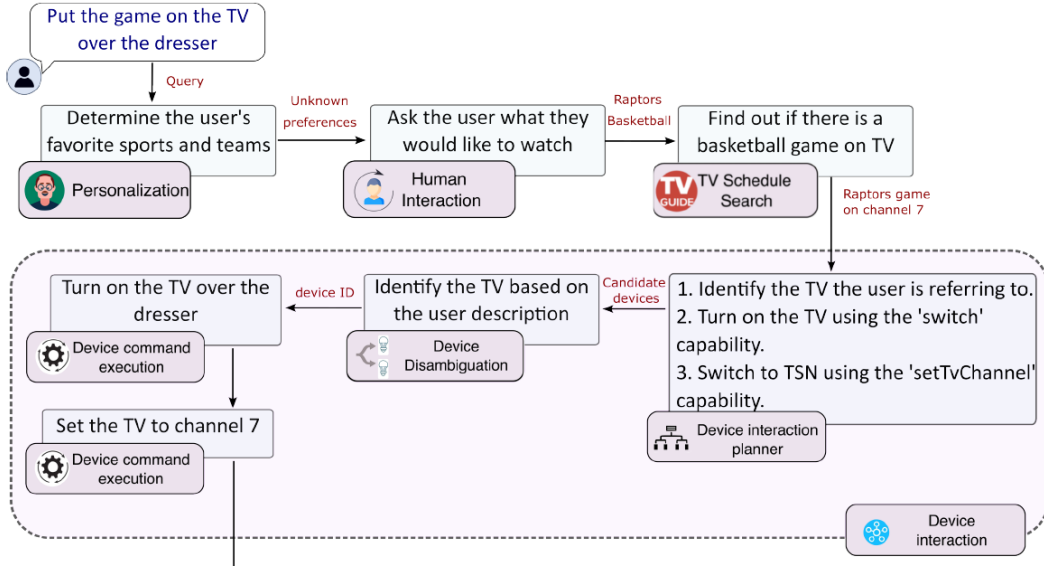


Figure 2.3: SAGE system execution flow (Adapted from: Rivkin et al. [16]). The central agent dynamically orchestrates multiple tools (Personalization, Human Interaction, TV Schedule Search, Device Interaction) to accomplish the user request. The dashed box represents the Device Interaction agent-tool, which generates a multi-step execution plan. Each tool selection is made autonomously by the LLM based on previous results, demonstrating centralized dynamic coordination.

The SAGE [16] system: Instead of a fixed pipeline, SAGE employs a single, top-level “agent-tool” that acts as a dynamic orchestrator [Abstract, 16]. As demonstrated in

its architecture and execution examples, this central agent dynamically decides the next step at every point in the process. It autonomously chooses which tool to call next (e.g., personalization, human_interaction, device_interaction) based on the real-time results of the previous step [16].

While the SAGE [16] system is a powerful demonstration of a dynamic, centralized agent, we can observe that managing such a stateful, step-by-step process is a very complex matter. This implementation complexity is evident in the custom logic the authors had to design, specifically their sequential decision-making loops, to manage the control flow [16].

Furthermore, this system is highly dependent on powerful LLMs. The SAGE authors note in their conclusion that to achieve acceptable performance, “today’s most powerful LLMs are needed” [16].

These observations: the high implementation complexity and the heavy dependency on top-tier models, motivated us to search for better tools to build multi-agent systems. This search directly led to the work in this thesis, which explores using LangGraph to build a hybrid architecture (Centralized task planning and dynamic device collaboration).

2.4 LangGraph: A Framework for Stateful Multi-Agent Systems

The challenges identified in existing systems, particularly the implementation complexity in SAGE [16], suggest the need for frameworks that can simplify multi-agent system development. LangGraph² is a framework developed by LangChain Inc. specifically to bridge the gap in orchestrating agents within stateful and concurrent environments [14].

LangGraph builds upon the core primitives of LangChain by introducing a graph-based orchestration layer implemented as a graph state machine. In this framework, the system is modeled as a stateful directed graph, where each node represents a computational step such as an LLM invocation or a tool execution, and edges define how control flows between these steps.

In contrast to traditional Directed Acyclic Graphs (DAGs), which impose a strictly linear execution order, LangGraph allows the construction of cyclic graphs. This design enables agents to return to earlier states for refinement, retries, or iterative reasoning, which is essential for supporting robust and autonomous workflows.

This architectural flexibility supports advanced dynamic routing capabilities for smart home scenarios. In implementation, device agents can use conditional logic to decide which agent should act next. For example, after a lighting agent completes its task, it can dynamically determine whether to route control back to the task planner or to another device agent based on the current task queue state. This is more flexible than having fixed edges between all possible device pairs.

The framework also handles state management automatically, eliminating the need for custom control flow logic like SAGE [16]. Additionally, LangGraph provides built-in checkpointing and fault tolerance mechanisms, allowing the system to handle edge cases gracefully: such as retrying a failed device control node without halting the entire system.

²<https://docs.langchain.com/oss/python/langgraph/overview>

The LangGraph provides four core characteristics³ that address key challenges in multi-agent system development:

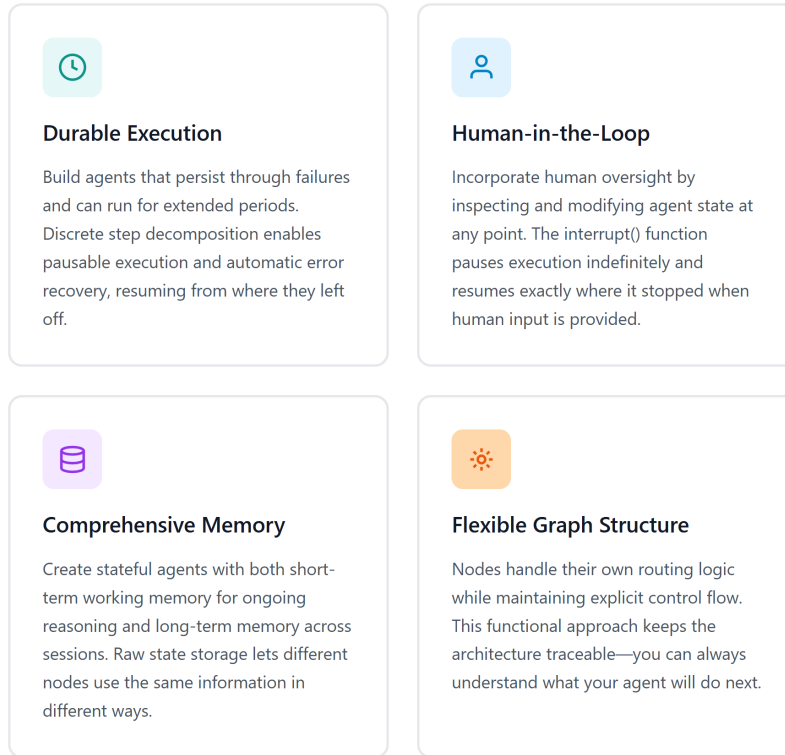


Figure 2.4: Characteristics of LangGraph framework (Source: LangChainDocs). The framework provides durable execution for persistent workflows, human-in-the-loop capabilities for oversight, comprehensive memory management, and flexible graph structures for traceable control flow.

These characteristics make LangGraph suitable for implementing the hybrid architecture proposed in this thesis. The dynamic routing capability is especially relevant because it allows task planning to remain centralized while device-level collaboration can adapt based on execution context.

2.5 Multi-Agent Coordination Theory

Multi-agent collaboration can be defined as Pure Coordination Settings. Within this theoretical framework, all home agents share aligned interests with no competitive incentives, and success hinges on how agents achieve perfect strategies through reasoning. This provides a theoretical model for collaboration among smart home devices. [2]

Tran et al. propose a comprehensive framework for understanding LLM-based multi-agent collaboration [17]. As shown in Figure 2.5, this framework emphasizes *collaboration channels* that facilitate coordination and orchestration among agents. These channels are

³<https://docs.langchain.com/oss/python/langgraph/overview>

characterized by their type (cooperation, competition, coopetition), structure (centralized, distributed, hierarchical), and strategy (rule-based, role-based, model-based), with coordination mechanisms at different stages (early, mid, late).

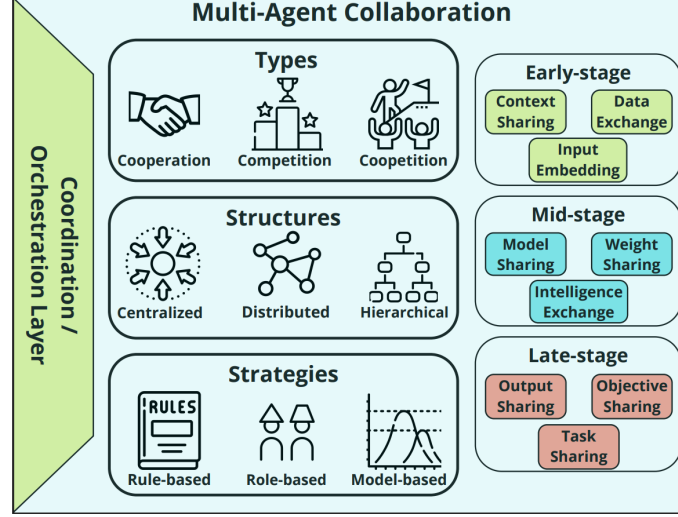


Figure 2.5: Framework for LLM-based multi-agent collaborative systems (Source: Tran et al. [17]). The framework focuses on collaboration channels defined by type, structure, and strategy, with coordination mechanisms at early, mid, and late stages. The framework accommodates diverse multi-agent systems under a unified structure.

Within this framework, the authors classify modern LLM-based agent collaboration into three core paradigms based on type: Cooperation, Competition, and Coopetition. Among these, the Cooperation mechanism requires agents to align their individual objectives with the system’s collective goals. This allows them to work together to solve complex tasks that are difficult for a single node to handle alone. Theoretically, this approach traces back to the classical Joint Intentions Theory. This theory emphasizes that collaboration is more than just synchronizing actions; it requires agents to establish a “Joint Persistent Goal” and a shared mental state [19].

In modern systems driven by Large Language Models (LLMs), this concept has evolved into role-based strategies. This involves assigning agents different expert roles, such as planners or executors, to improve the overall efficiency of the system. While Competition and Coopetition mechanisms are effective in scenarios like debates or negotiations where opposing views can stimulate deeper reasoning, the Smart Home context is different. In a Smart Home, the primary need is for seamless coordination and rapid response among heterogeneous devices, such as lighting, temperature control, and search engines, rather than managing conflicting interests. Therefore, focusing on this specific application, this study concentrates on the Cooperation mechanism between device agents. It aims to explore how to leverage SharedPlans Theory and Joint Intentions Theory to achieve efficient coordination among household devices [19].

3

System Design and Implementation

This chapter presents the design and implementation of a hybrid multi-agent system for smart home. The system enables users to control home devices through natural language commands, with multiple LLM-based agents that can dynamically collaborate to execute tasks.

3.1 Overall Architecture and Components

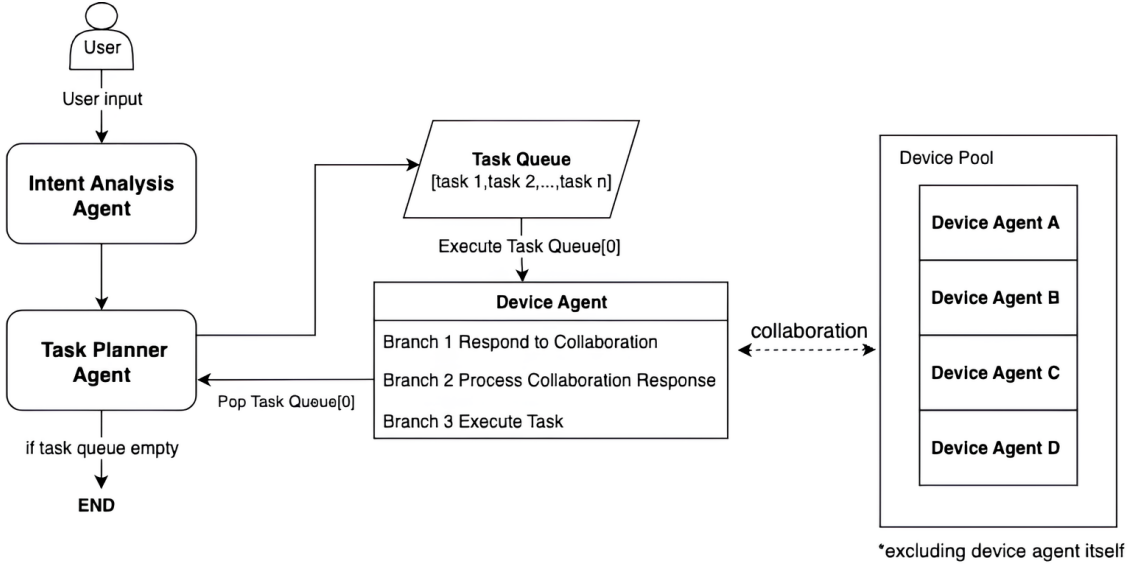


Figure 3.1: System Architecture Overview

The overall system architecture is designed to facilitate robust and intelligent smart home task execution. As illustrated in Figure 3.3, the system comprises four components: a **Human Node**, an **Intent Analysis**, a **Task Planner**, and multiple **Device Agents**.

The dashed line illustrates an example of a dynamic collaboration path. This design enables flexible task execution without predefined collaboration patterns, as agents make collaboration decisions based on task requirements rather than pre-programmed rules.

Components

- **Human Node:** Serves as the system’s interface for receiving user commands in natural language. It captures user input and forwards it to the Intent Analysis for processing.
- **Intent Analysis Agent:** Analyze user input by calculating independent information points or intent points to measure user input complexity, and extract keywords such as time and location for reference by the task planner agent.
- **Task Planner Agent:** Analyzes the original user input to understand user needs and generates a high-level task queue. It assigns initial tasks to the appropriate device agent based on the agent’s capabilities and the intent of the user request.
- **Device Agents:** Specialized agents that simulate specific smart home devices, including information devices (Clock, Calendar, Search Engine, Fridge), execution devices (Lighting, Thermostat, Audio System), and a display device (TV Display). Each agent can execute assigned tasks independently or initiate collaboration when needed.

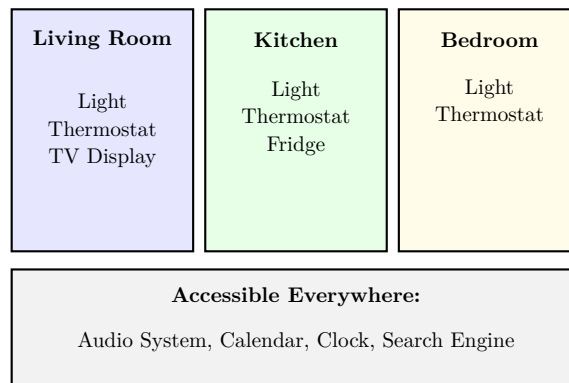


Figure 3.2: Device types in a typical smart home environment. The system implements eight device agents in total, with one agent per device type simulating its device operations across all locations.

Figure 3.2 shows a typical smart home scenario with example device placements across different rooms. The system implements eight device agents in total—one for each device type. Each agent simulates operations for its device type across all mentioned locations. For example, a single Light Agent handles commands like ‘turn off kitchen light’ or ‘dim bedroom light’ by processing the location information in user requests.

3.2 System Workflow

After user input is received by the human node, it is analyzed by the Intent Analysis Agent and Task Planner Agent, resulting in a task queue composed of devices and task requirements. The devices agent then proceed to execute the tasks. Tasks are executed sequentially from the task queue. After each task completes, control returns to the Task Planner, which proceeds to the next task in the queue.

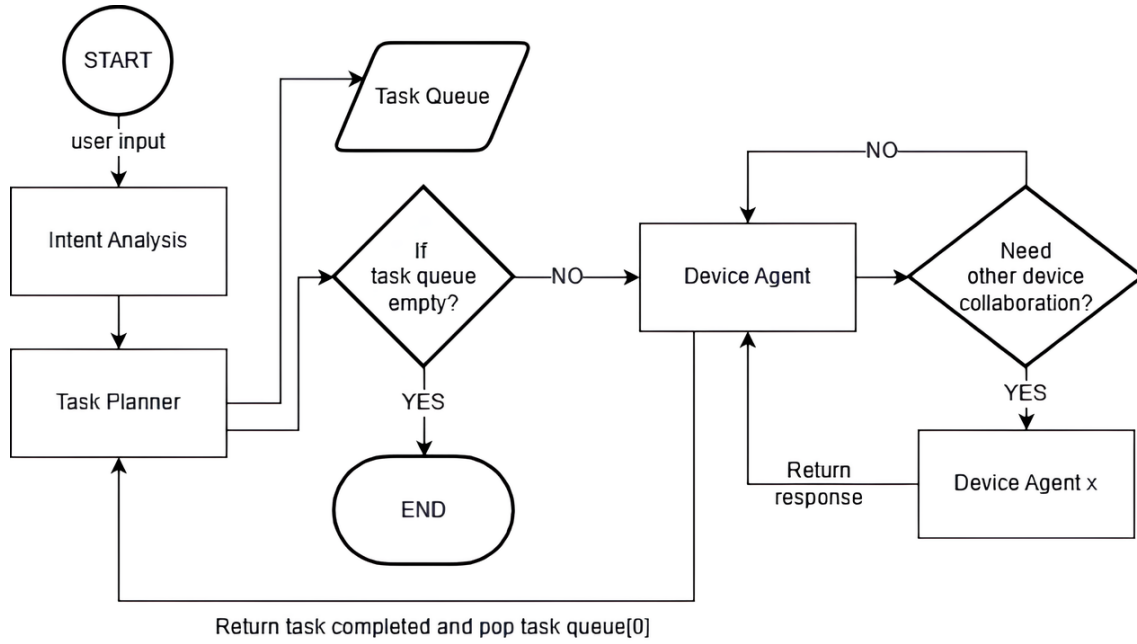


Figure 3.3: System Workflow

Sequential Task Processing

The Task Planner generates a task queue containing multiple tasks that need to be executed sequentially. Each task in the queue specifies a target device and the action to be performed:

```

Task queue: [
    {"device1","action1"}, #Task_1
    {"device2","action2"}, #Task_2
    ...,
    {"devicen","actionn"} #Task_n
]
  
```

The execution follows this pattern:

```

Task Planner → [Execute Task 1] → Task Planner →
                [Execute Task 2] → Task Planner → ... → END
  
```

After each task completed, control returns to the Task Planner agent, which proceeds to the next task. This ensures sequential processing of all tasks in the queue.

Task Execution: Independent or Collaborative

Each task can be executed in two ways:

Independent Execution

When an agent can complete its task without external assistance:

```

Task Planner → Device Agent → Task Planner
  
```

Example: Clock agent executing “Wake me up at 7am”

1. Analyzes task requirements
2. Determines no external capabilities needed
3. Executes: “Alarm set for 7 am tomorrow”
4. Router return to Task Planner

Collaborative Execution

When an agent requires external capabilities:

Task Planner → Agent A → Agent B → Agent A → Task Planner
(request) (respond) (complete)

Agent A directly communicates with Agent B during task execution, without returning to the Task Planner. This reduces coordination overhead while maintaining sequential task processing.

Example: Lighting Agent executing "dim lights gradually at 10pm"

1. Lighting analyzes task requirements
2. Determines it needs clock's capability
3. Requests collaboration from Clock agent
4. Clock responds with information which Lighting Agent needed
5. Lighting completes task using the response
6. Router to Task Planner

In both cases, the collaboration decision is made dynamically by the executing agent using LLM-based analysis, not predetermined by the Task Planner. The detailed decision mechanism is presented in Section **Dynamic Device Collaboration**.

3.3 Design Rationale

This section discusses three architectural decisions: the hybrid coordination approach, the choice of LangGraph framework, and the single-round collaboration

Hybrid Architecture Design

Purely centralized architectures (such as the SAGE [16] system) place all coordination decisions under a single orchestrator. While this architecture provides powerful global control capabilities, it requires the central agent to predict all possible device interactions, thereby increasing decision complexity and reasoning overhead. Instead, fully decentralized systems have the risk of coordination failure when devices lack a shared understanding of user intent.

Our hybrid approach achieves a separation of responsibilities: the task planner decomposes high-level tasks and identifies required devices without predicting collaborations; device agents make collaborative decisions based on specific task requirements. This separation reduces the complexity of the task planner while preserving coordination flexibility at the device level.

LangGraph Framework Selection

The choice of LangGraph addresses implementation complexity. SAGE [16] required custom state management logic to orchestrate multi-step agent workflows. LangGraph provides several advantages:

Built-in State Management: The framework handles state persistence and updates automatically through Command objects, eliminating the need for custom control flow logic.

Dynamic Routing: Command objects allow agents to specify the next node at runtime through the goto parameter, enabling flexible collaboration patterns without predefined connections between all device pairs.

Graph Abstraction: The graph-based structure naturally represents agent workflows, making the system architecture more maintainable and easier to extend with new device agents.

These capabilities significantly reduce implementation complexity compared to custom orchestration logic.

Single-Round Collaboration

The system limits device agents to single-round collaboration: each agent can request help from one other agent during task execution. This constraint prevents potential problems while keeping the implementation simple.

Without this limitation, agents could form endless loops (Agent A asks Agent B, which asks Agent C, which asks Agent A again). Single-round collaboration prevents this by ensuring that agents responding to requests (Branch 1) cannot make their own collaboration requests. The three-branch structure works well under this constraint.

In practice, single-round collaboration is sufficient for most common smart home scenarios. Typical coordination between devices (getting the current time, checking calendar events, requesting search recommendations) requires only one collaboration step.

3.4 Implementation Framework

The system is implemented using LangGraph⁴, a framework for building stateful multi-agent applications with LLMs. This section introduces the framework components and implementation patterns.

Framework Components

LangChain Base Components

LangGraph is built on LangChain, leveraging its core components for LLM interaction. Our implementation uses:

- PromptTemplate: A component for creating reusable prompt templates with variable substitution. Enables consistent prompt structure across agents while allowing dynamic content injection.

⁴<https://langchain-ai.github.io/langgraph/>

- JsonOutputParser: A parser that automatically generates JSON format instructions and extracts structured data from LLM responses. Handles common output inconsistencies (explanatory text, markdown code blocks).
- LCEL (LangChain Expression Language): A declarative syntax for composing LLM chains using the pipe operator (`|`). The pattern `prompt | llm | parser` creates a data flow from input to structured output.

LangGraph Extensions

LangGraph extends LangChain with stateful multi-agent capabilities:

- Graph-based Execution: Agents are nodes in a graph with conditional routing between nodes
- Shared State: A typed state object accessible to all agents, enabling coordination through state updates
- Dynamic Routing: Command objects⁵ specify next agent and state updates, enabling flexible control flow

Language Model Configuration

All agents use `google/gemma-2-9b`⁶, a lightweight open-source language model deployed locally via Ollama⁷. The model is configured with zero temperature (`temperature=0.0`) to ensure deterministic and consistent outputs across all agents.

Rather than using different models for different agents, specialization is achieved through customized prompts tailored to each agent’s specific device capabilities and responsibilities. This unified model approach simplifies deployment while maintaining system independence and data privacy through local execution.

State Management

The system maintains a centralized state throughout execution, implemented as a `TypedDict` structure called `SmartHomeState`.

Key state fields include:

- `messages`: Conversation history for context
- `original_user_input`: The complete user input
- `complexity_score`: Measured input complexity
- `infos`: Analyzed information points or intents
- `key_modifiers`: Key modifiers in information points
- `task_queue`: Ordered list of tasks to be executed
- `collaboration_request`: Active collaboration requests between devices
- `task_history`: Save results from devices that have been executed or collaborated on
- `pending_task`: Tasks awaiting collaboration responses
- Device-specific result fields (e.g., `lighting_result`, `clock_response`)

⁵ <https://docs.langchain.com/oss/python/langgraph/graph-api#command>

⁶ <https://huggingface.co/google/gemma-2-9b>

⁷ <https://docs.ollama.com/>

All nodes in the system read from and update this centralized state, when an agent completes a task or responds to a collaboration request, it updates the relevant state fields through the update field in their returned Command objects, enabling information flow between agents.

Prompt Engineering and Output Parsing

LangChain components (PromptTemplate, JsonOutputParser, and LCEL) introduced in the previous subsection, this subsection presents the implementation pattern used across all agents.

```
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import PromptTemplate

parser = JsonOutputParser()
prompt = PromptTemplate(
    template="""
    Generate a task queue in JSON format.
    {format_instructions}

    User input: {user_input}
    """,
    input_variables=["user_input"],
    partial_variables={
        "format_instructions": parser.get_format_instructions()
    }
)

chain = prompt | llm | parser
result = chain.invoke({"user_input": "I want to sleep"})
```

This pattern enables dynamic content injection, automatic JSON format instructions, and reliable output parsing across all agents.

Graph Construction

The system is structured as a directed graph using LangGraph's StateGraph. Each component (Human Node, Intent Analysis, Task Planner, and Device Agents) is added as a node in the graph. The graph begins with the Human Node, which uses an interrupt mechanism to capture user input, then follows a fixed path through Intent Analysis to the Task Planner. After the Task Planner, all subsequent routing is controlled dynamically through Command objects returned by agents. This dynamic routing forms the foundation for device-to-device collaboration, allowing agents to route directly to each other based on task requirements.

The graph is compiled with a MemorySaver checkpoint, enabling state persistence across the interrupt point where user input is captured.

Execution Flow Control

Using the components introduced in the previous subsection, execution flow is managed through Command objects returned by each node:

```
return Command(
    update={
        "task_queue": ..., # State fields to modify
        "collaboration_request": {...}
    },
    goto="next_agent" # Router to the next node to execute
)
```

This mechanism enables dynamic routing: agents update state fields and specify the next node based on runtime conditions.

3.5 Centralized Task Planning

The centralized task planning consists of two sequential modules: Intent Analysis Agent and Task Planner Agent. This section details their design principles and implementation mechanisms.

Intent Analysis and Complexity Quantification

Design Objective

The Intent Analysis Agent preprocesses user input by decomposing multi-requirement sentences into distinct information units and extracting key modifiers such as time specifications or location markers. It also assesses input complexity. These results serve as additional references for the Task Planner, which conducts its own analysis of the original user input to generate device-specific tasks.

Information Point Definition

We define an *information point* as:

One intent, one feeling, or one fact

Each information point should be complete and clear, including all relevant details (what, how, when, why) that make it independently understandable.

User Input	Count	Key modifiers	Rationale
“turn on the light for 30 minutes”	1	for 30 minutes	One intent
“make room cozy for reading”	1	for reading	One complete goal
“turn the lights up to full brightness and play gentle music ”	2	full brightness, gentle	Two distinct intents
“I’m cold and want to be warm”	2	None	Feeling + intent

Table 3.4: Examples of information pointS for different input types

Complexity Quantification

The complexity score is calculated as:

$$\text{complexity_score} = |\text{infos}| \quad (3.1)$$

Analysis Example

Consider the following complex natural language input:

Input: “I just got home from work and I’m feeling exhausted. The weather forecast says it’s going to be cold tonight. I want to create a cozy atmosphere for relaxation.”

Extraction Result:

infos: [I just got home from work and I’m feeling exhausted, The weather forecast says it’s going to be cold tonight, I want to create a cozy atmosphere for relaxation]

key_modifiers: [exhausted, cold, cozy]

Complexity Score: 3

Implementation

The Intent Analysis Agent uses an LLM-based approach with structured prompting to identify information points. The implementation process consists of four steps:

1. Construct a prompt containing information point definition, extraction rules, and few-shot examples
2. Pass the user message and prompt to the LLM for processing
3. Parse the LLM’s JSON output to extract the list of information points and key modifiers
4. Calculate the complexity score as the count of extracted points

The prompt design emphasizes three key principles to ensure consistent extraction:

- **Semantic independence:** Each extracted point must be self-contained and interpretable without additional context. For example, “feeling exhausted” is a complete emotional state, while “exhausted” alone lacks semantic completeness.
- **Keep details:** Modifying phrases remain attached to their actions rather than being extracted separately. For instance, “play music at low volume” constitutes one information point, not two, as “at low volume” specifies how to execute the action rather than representing an independent need.

The LLM outputs a JSON structure containing the extracted information points and key modifiers:

```
{
  "infos": ["info1", "info2", ..., "infn"],
  "key_modifiers": ["modifier1", "modifier2", ..., "modifiern"]
}
```

This structured output serves as a reference for the task planner while also quantifying the complexity of user input.

Task Generation and Distribution

Design Objective

The Task Planner Agent analyzes the complete user input to understand explicit and implicit requests, then generates an ordered high-level task queue which means no overthinking about collaboration. The Task Planner must interpret user intent and make device assignment decisions.

The Task Planner addresses several key challenges:

- **Device capability matching:** Selecting appropriate devices based on their functional capabilities
- **Avoiding overthinking:** Creating a high-level task queue without preemptively assigning collaboration partners

Implementation

The Task Planner operates based on a three-branch conditional structure that determines its behavior according to the current system state:

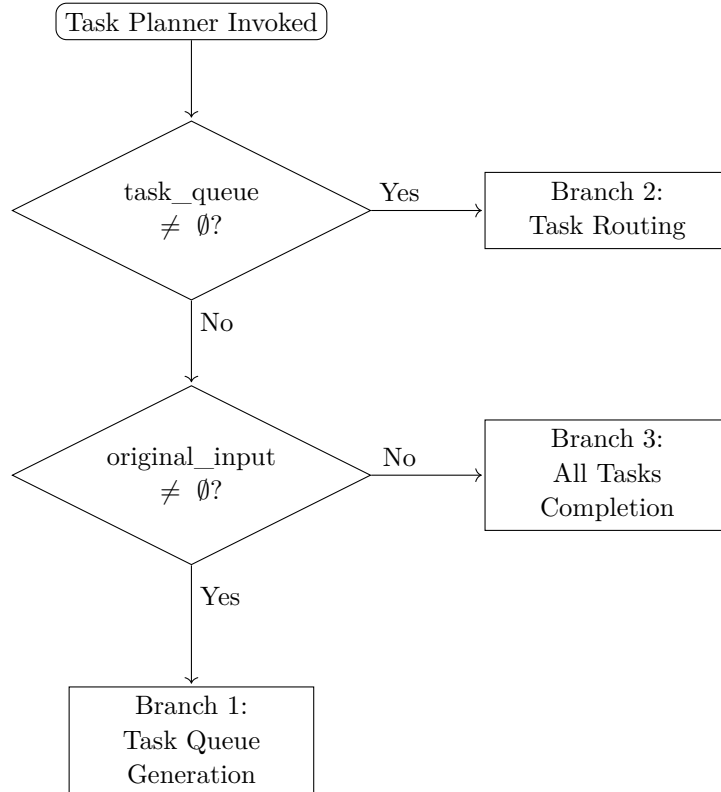


Figure 3.5: Task Planner Decision Flow

Branch 1: Task Queue Generation

Trigger Condition: $\text{task_queue} = \emptyset \wedge \text{original_user_input} \neq \emptyset$

When receiving a new user input (with empty task queue), the Task Planner generates a complete task queue:

1. Invoke the LLM to analyze user intents and generate task queue(assignments)
2. Parse the JSON output to get the task queue
3. Assign the first task to its designated device agent
4. Clear the original_user_input field to prevent re-generation of task queue

Prompt Engineering

The task planner generates a task queue through structured prompt sentences, putting the results of intent analysis into the prompt as part of the reference. This process invokes the LLM, which includes:

Device	Primary Capabilities
<i>Information & Display</i>	
clock	Time queries, set alarm or timer
calendar	Schedule, event, reminder
search engine	Information lookup, recommendations
tv display	Display or show visual content
fridge	Food inventory
<i>Execution Devices</i>	
lighting	Illumination control, color adjustment
thermostat	Temperature control
audio system	Sound playback, volume control

Table 3.6: Device capability

1. **Device capability catalog:** A comprehensive tabular 3.6 of all available devices and their functions, categorized as execution devices (lighting, thermostat, audio system), information devices (clock, calendar, fridge, search engine), and display device (TV display).
2. **Task assignment principles:** Assign user intent to the most appropriate devices. Do not place devices requiring collaboration into the task queue. Task requirements must include contextual information to prevent incomplete device understanding.
3. **Few-shot examples:** Showcasing a typical example of proper task decomposition, such as:
 - Simple requests: “turn on lights” → single lighting task
 - Multi-device scenarios: “I’m cold and tired” → Multiple ambiance-creating devices
 - User input with context: “It’s getting late. I need to sleep.” → sleep environment setup

The output format is a JSON task queue where each task specifies a target device and a natural language action description. For example:

```
{
  "task_queue": [
    {"device": "lighting", "action": "dim lights for sleep"},
    {"device": "thermostat", "action": "set cool temperature"}
  ]
}
```

Branch 2: Task Routing**Trigger Condition:** $\text{task_queue} \neq \emptyset$

When tasks remain in the queue, the Task Planner acts as a sequential router:

1. Retrieve the first task from the queue: $\text{current_task} \leftarrow \text{task_queue}[0]$
2. Route to the corresponding device agent based on the task's device field

This branch ensures sequential task queue execution.

Branch 3: All Tasks Completion**Trigger Condition:** $\text{task_queue} = \emptyset \wedge \text{original_user_input} = \emptyset$

When the state fields task queue and user input are both empty, all tasks have been completed. The Task Planner terminates execution by routing to the END state, concluding the workflow.

Planning Example

"I just got home from work and I'm feeling exhausted. The weather forecast says it's going to be cold tonight. I want to create a cozy atmosphere for relaxation."

Generated Task Queue:

1. {device: "lighting", action: "create a cozy and relaxing lighting environment"}
2. {device: "thermostat", action: "set a warm temperature for the evening"}
3. {device: "audio_system", action: "'play calming music for relaxation"}

3.6 Dynamic Device Collaboration Implementation

This section presents the implementation of dynamic device collaboration. Each device agent determines its behavior through a three-branch decision structure based on the current system state.

Branch 1 (Highest Priority): Responding to Collaboration Requests

When an agent receives a collaboration request from another agent, it processes the request using its capabilities and returns a response.

Branch 2 (Second Priority): Handling Collaboration Responses

When an agent has a pending task awaiting collaboration information, it combines the received response with the original task to complete execution.

Branch 3 (Lowest Priority): Processing New Tasks

When an agent receives a new task from the Task Planner, it uses LLM-based reasoning to determine whether the task can be completed independently or requires collaboration with other agents.

Figure 3.7 illustrates this priority-based decision structure.

The following sections detail each branch's implementation, state management mechanisms, and a complete collaboration example using the bedtime routine scenario.

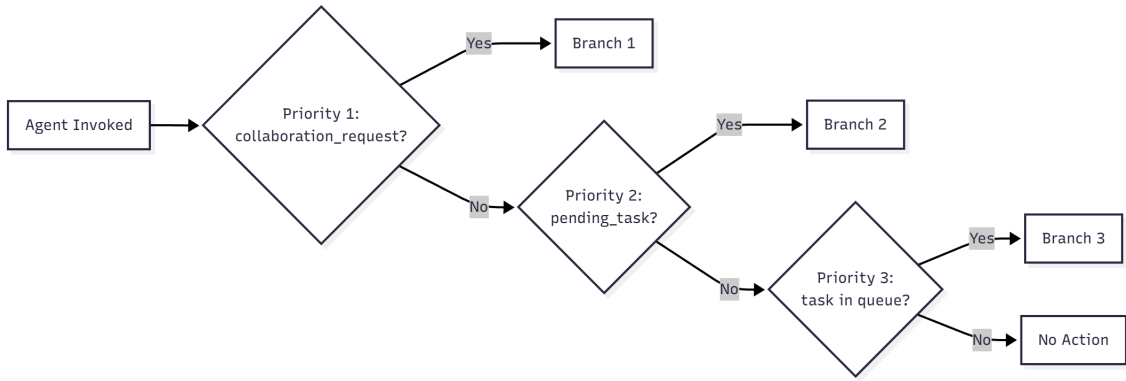


Figure 3.7: Three-Branch Decision Structure

Three-Branch Decision Structure

Each device agent implements the three-branch structure using `if-elif-elif` conditional logic, ensuring that only one branch executes per invocation. The branch selection is state-driven: the agent examines `collaboration_request`, `pending_task`, and `task_queue` in order, executing the first branch whose trigger condition is satisfied.

Branch 1: Responding to Collaboration Requests

Trigger Condition: When this agent receives a collaboration request from another agent.

Figure 3.8 illustrates the execution flow for Branch 1.



Figure 3.8: Branch 1: Responding to Collaboration Requests

When an agent receives a collaboration request from another agent, it acts as a collaboration provider. The agent first extracts the requester’s identity and the specific request content from the `collaboration_request` field. Then it uses LLM-based reasoning, combined with its domain-specific capabilities, to generate an appropriate response. The response is stored in a designated state field specific to the responding agent (e.g., `clock_response` for the Clock Agent, `lighting_response` for the Lighting Agent). Finally, the agent clears the `collaboration_request` field and routes control back to the requesting agent using a Command object.

Critical design principle: Branch 1 responders must operate independently and cannot initiate further collaborations. This constraint is enforced through prompt design, preventing unbounded collaboration chains and ensuring workflow termination.

Branch 2: Handling Collaboration Responses

Trigger Condition: When this agent has a pending task awaiting collaboration information.

Figure 3.9 illustrates the execution flow for Branch 2.



Figure 3.9: Branch 2 Execution Flow

Branch 2 is used to complete tasks that were previously suspended while awaiting collaboration information. When an agent receives a collaboration response from another agent, it combines this response with the original task to complete the execution.

The agent first retrieves the collaboration response from the corresponding state field (e.g., `clock_response` if waiting for the Clock Agent’s response). Then it retrieves the original task action from the `pending_task` field. Next, the agent uses LLM-based reasoning to combine the collaboration response with the original task information, generating the final task result.

After task completion, the agent removes the task from `task_queue` (the first task at the top of the queue), clears both the `pending_task` field and the collaboration response field, and routes control back to the Task Planner using a Command object.

Branch 3: Processing New Tasks

Trigger Condition: When the first task in the task queue targets this agent.

Figure 3.10 illustrates the execution flow for Branch 3.

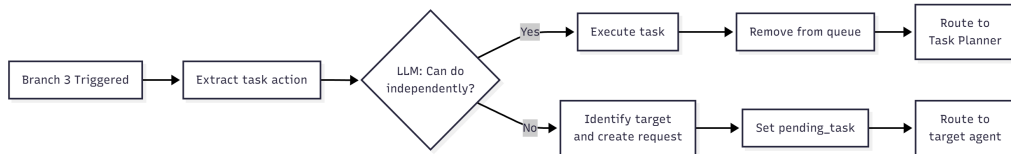


Figure 3.10: Branch 3 Execution Flow

Branch 3 best demonstrates the dynamic device collaboration capability. When an agent receives a new task from the Task Planner, it uses LLM-based analysis to determine whether the task can be completed independently or requires assistance from other agents.

The agent first extracts the task action from the task queue. It then performs an LLM-based analysis with three key inputs: (1) the agent’s own capabilities explicitly defined in the prompt, (2) a list of other available agents with their functions, and (3) the task history containing results from previously executed agents. By analyzing these inputs, the LLM determines whether the task can be completed independently. The task history is particularly important as it allows the agent to check if required information has already been obtained by other agents, thereby avoiding redundant collaboration requests.

If the task can be completed independently: The agent executes the task using its capabilities and available information from the task history, generates the task result, removes the task from the queue, and routes control back to the Task Planner using a Command object.

If collaboration is needed: The agent identifies what information is required and which agent can provide it. It creates a collaboration request, stores the current task in the `pending_task` field for later resumption, and routes control to the target agent using a Command object. This decision is made only after verifying that the needed information is not already available in the task history.

Collaboration State Management

The three-branch mechanism uses three dedicated state fields to coordinate collaboration: `collaboration_request`, `pending_task`, and device-specific response fields. These fields enable agents to track collaboration status and determine which branch to execute.

This section explains each field’s purpose, structure, and lifecycle.

Collaboration_request Field

The `collaboration_request` field stores collaboration request information between agents. When an agent processes a new task and the LLM determines that the task cannot be completed independently, the agent creates this field to initiate a collaboration request.

The field structure contains three key components:

- `requester`: The name of the agent initiating the collaboration request
- `target`: The name of the target agent being requested to collaborate
- `request`: The specific collaboration request content(in natural language)

The field is created when processing new tasks (Branch 3). Control is then routed to the target agent, which processes the request (Branch 1) and clears this field after generating a response, indicating that the collaboration request is complete.

Task_history Field

The `task_history` field is a list of dictionaries that functions as a sequential log. It records all information generated by participating agents during the execution of a task, tracking every action, response, and collaboration step to build a complete record of the task queue’s lifecycle.

Each dictionary entry appended to the list contains the following key components:

- `device`: The name of the agent that generated this entry.
- `type`: The type of the entry, such as `task_completion`, `collaboration_request`, or `collaboration_response`.
- `action_taken`: The specific task or request the agent was processing that resulted in this entry.
- `result`: The output, data, or response generated by the agent for that action.

This field is updated cumulatively. After an agent completes its designated action (either an independent task or responding to a collaboration), a new entry dictionary is appended to the `task_history` list. This provides all subsequent agents in the workflow with a complete contextual record of previously executed actions and their results.

Pending_task Field

The `pending_task` field stores tasks that are suspended while awaiting collaboration information. When an agent determines it needs assistance from another agent, it stores the current task in this field to enable resumption after receiving the collaboration response.

The field structure contains three key components:

- `device`: The name of the agent with the pending task
- `action`: The original task action description
- `waiting_for`: The name of the agent whose response is being awaited

The field is created when an agent initiates a collaboration request (Branch 3), at the same time as `collaboration_request`. After the agent receives the collaboration response and completes the original task (Branch 2), this field is cleared, indicating that the pending task is finished.

Device Response Fields

Each device agent has a dedicated response field to store the information it provides during collaboration. These fields follow the naming convention `{agent_name}_response`, such as `clock_response`, `lighting_response`, and `calendar_response`.

Dedicated response fields prevent one agent's response from overwriting another's when multiple collaborations occur. This design also makes it clear which agent provided each piece of information.

The lifecycle of these fields is as follows: a responding agent creates the field when processing a collaboration request (Branch 1), the requesting agent reads the field to obtain collaboration information (Branch 2), and clears the field after completing the task.

3.7 Complete Example: Bedtime Routine

This section shows the complete collaboration mechanism through the bedtime routine scenario. We trace the system state changes and agent interactions throughout the entire workflow.

"I'm trying to establish a bedtime routine. I want the lights to dim gradually starting at 10pm. Also play some music to help me sleep."

After the Task Planner processes this input, the task queue contains:

```
task_queue = [
    {"device": "lighting", "action": "dim the lights gradually starting at 10pm"},
    {"device": "audio_system", "action": "play some music to help me sleep"}
]
```

3 System Design and Implementation

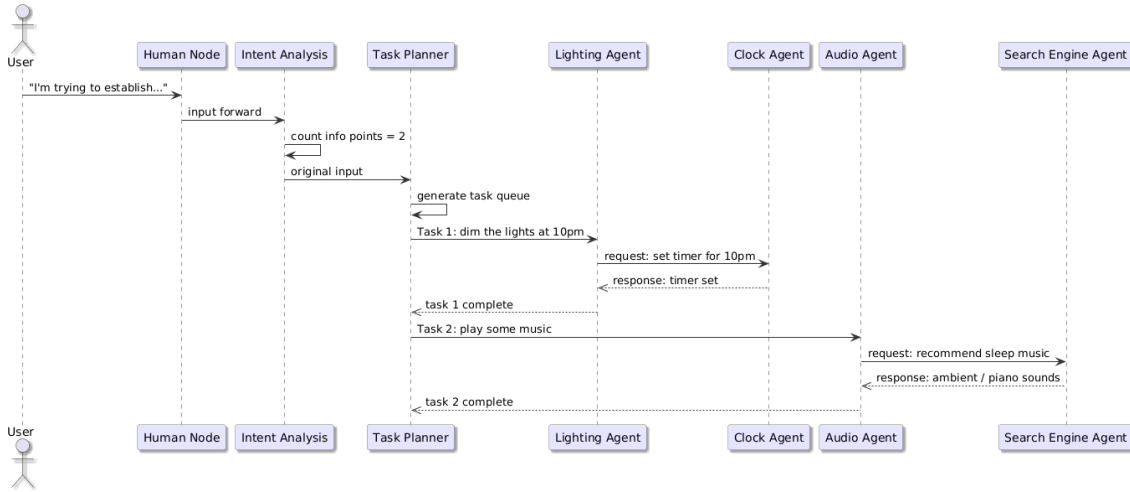


Figure 3.11: Sequence diagram of the bedtime routine

The following steps detail how agents collaborate to complete these tasks.

Step 1: Lighting Agent Initiates Collaboration

The Task Planner routes control to the Lighting Agent. At this point, both `collaboration_request`, `task_history` and `pending_task` are empty, while the first task in `task_queue` targets the lighting agent. Therefore, the Lighting Agent enters the new task processing branch (Branch 3).

The agent extracts the task action: “dim gradually starting at 10pm”. Through LLM analysis, the agent determines it can execute the dimming operation, but “starting at 10pm” requires time management capabilities. Therefore, it decides to request collaboration from the Clock Agent, asking for a timer at 10pm to trigger the dimming.

The Lighting Agent updates the system state:

```
collaboration_request = {
    "requester": "lighting",
    "target": "clock",
    "request": "Set a timer to start dimming the lights at 10:00 PM"
}
```

```
pending_task = {
    "device": "lighting",
    "action": "dim the lights gradually starting at 10pm",
    "waiting_for": "clock"
}
```

```
task_history =[
    {'device': 'lighting',
     'type': 'collaboration_request',
     'action_taken': 'dim the lights gradually starting at 10pm',
```

```
'result': {'target': 'clock',
           'request': 'Set a timer to start dimming the lights at 10:00 PM'}
}]
```

Control is then routed to the Clock Agent.

Step 2: Clock Agent Responds to Collaboration Request

The Clock Agent receives control and finds that `collaboration_request` targets it. Therefore, the Clock Agent enters the collaboration response branch (Branch 1).

The agent processes the request from `collaboration_request`: “Set a timer to start dimming the lights at 10:00 PM”. Using time management capabilities, the Clock Agent generates a response showing that the timer has been set.

The Clock Agent updates the system state:

```
clock_response = "Timer set to start dimming lights at 10:00 PM."
collaboration_request = {} # Cleared
```

Control is then routed back to the Lighting Agent (the requester).

Step 3: Lighting Agent Completes Pending Task

The Lighting Agent receives control again and finds that `collaboration_request` has been cleared, but `pending_task` contains an unfinished task for this agent. Therefore, the Lighting Agent enters the pending task completion branch (Branch 2).

The agent retrieves the collaboration response from `clock_response` and the original task from `pending_task`. It then combines both information: the original requirement to “dim gradually” and the Clock Agent’s response that a timer has been set for 22:00. Using this combined information, the agent completes the task.

The Lighting Agent updates the system state:

```
lighting_result = "Waiting for timer to start at 10:00 PM to begin dimming
lights gradually."
```

```
task_queue = [
    {"device": "audio_system", "action": "play some music to help me sleep"}
] # First task removed
```

```
pending_task = None # Cleared
```

```
clock_response = None # Cleared
```

```
task_history = task_history + [new_entry] # update task history
```

Control is then routed to the Audio_system.

Step 4: Audio System Agent Initiates Collaboration

The Task Planner finds that `task_queue` still contains a remaining task, so it routes control to the Audio System Agent (the device specified in `task_queue[0]`).

The Audio System Agent enters the new task processing branch (Branch 3). It extracts the task action: “play some music to help me sleep.” Through LLM analysis, the agent determines it can execute the play music operation, but “some music” requires `search_engine` capabilities. Therefore, it decides to request collaboration from the `Search_engine` Agent, asking for recommend music for sleeping.

```
collaboration_request = {
    "requester": "audio_system",
    "target": "search_engine",
    "request": "recommend music for sleeping"
}

pending_task = {
    "device": "audio_system",
    "action": "play some music to help me sleep",
    "waiting_for": "search_engine"
}

task_history = task_history + [new_entry] #update task history
```

Control is then routed to the `Search_engine` Agent.

Step 5: Search_engine Agent Responds to Collaboration Request

The `Search_engine` Agent receives control and finds that `collaboration_request` targets it. Therefore, the `Search_engine` Agent enters the collaboration response branch (Branch 1). The agent processes the request from `collaboration_request`: “recommend music for sleeping”. Using its ‘look up information’ capabilities, the `Search_engine` Agent generates a response.

The `Search_engine` Agent updates the system state:

```
search_engine_response = "Ambient music, classical piano, nature sounds,
white noise, binaural beats"

collaboration_request = {} # Cleared
task_history = task_history + [new_entry] # update task history
```

Control is then routed back to the `Audio_system` Agent (the requester).

Step 6: Audio_system Agent Completes Pending Task The Audio System Agent updates the system state:

```
audio_system_result = "Playing ambient music to help you sleep."
```

```
pending_task = None # Cleared
search_engine_response = None
task_history = task_history + [new_entry] # update task history
task_queue = [] # All tasks completed
```

Control is then routed back to the Task Planner.

Example Completion

The Task Planner finds that `task_queue` is now empty, indicating that all tasks have been completed. The workflow terminates successfully, with both the lighting schedule and audio playback configured according to the user’s bedtime routine request.

Example Analysis

This example shows how agents make independent collaboration decisions based on task requirements. The Lighting Agent identified its need for time information and requested help from the Clock Agent, The audio system agent has no content to play, so it requests the search engine agent to provide recommendations. This selective behavior emerges from LLM-based reasoning rather than pre-programmed rules.

4

Benchmark

4.1 Introduction

Evaluating LLM-based multi-agent systems requires systematic benchmarks that assess agent capabilities across diverse scenarios. Recent benchmarks have evaluated agents in domains such as research collaboration, software development, and web navigation [22], while systematic surveys have categorized evaluation approaches along dimensions including agent behavior, capabilities, and assessment methods [12]. However, these benchmarks primarily target general-purpose tasks and lack domain-specific evaluation for smart home scenarios.

Smart home systems present unique evaluation requirements. User inputs are expressed in natural language with varying intent complexity, ranging from simple single-device commands to complex multi-step requests. Multiple device agents must coordinate to complete tasks, and task correctness can be verified by checking the invoked devices. These characteristics demand a benchmark that systematically evaluates performance across complexity levels.

To address this gap, this chapter presents a benchmark specifically designed for multi-agent smart home systems. The benchmark consists of 60 test cases organized by intent complexity into three groups: simple (1-2 intents), moderate (3 intents), and complex (4 and more intents), with 20 cases in each group. All test cases reflect realistic smart home scenarios and involve one or more of the eight device agents: Calendar, Clock, Search Engine, Light, Thermostat, Audio system, TV Display, and Fridge. The benchmark also defines evaluation criteria for assessing task generation, device collaboration, and overall completion.

4.2 Design Methodology

The number of intents serves as a natural metric for measuring task complexity in smart home scenarios. Simple requests typically involve one or two intents—such as “Turn on the lights” (single intent) or “I’m hungry” (querying recipes, possible fridge collaboration). As users express more complex needs, the number of intents naturally increases. For example “Find me a lasagna recipe, check if I have ground beef in the fridge, show the instructions

on the screen, and set a timer for 1 hour” involves four distinct intents.

Additionally, the number of devices involved reflects the task complexity. Therefore, beyond grouping based on the number of intents, the 60 test cases can be further analyzed according to the number of devices involved. These two dimensions: number of intents and number of devices, provide complementary views for a systematic evaluation of system performance.

Intent Complexity

Based on an analysis of typical smart home scenarios, we have defined three complexity levels:

Simple (1-2 intents): These cases represent straightforward requests involving one or two distinct user intents. Examples include single device operations (“Set a 20 minutes timer for my laundry”), simple queries (“Are there any barbershops nearby?”), or requests combining query and action (“What time does my next schedule begin?” involves checking calendar and clock). These cases typically require minimal device coordination.

Moderate (3 intents): These cases involve three distinct intents requiring coordination across multiple devices. Examples include “I need to study for 45 minutes, brighten the light and play some focus music” (set timer + adjust light + play music) or “turn on the light in the living room, show me some TV shows and set a one-hour countdown timer”. These cases represent common multi-step scenarios in smart home.

Complex (4 intents): These cases represent complex scenarios with four or more intents, often involving conditional logic, sequential steps, or comprehensive scene preparation. Examples include “Find me a lasagna recipe, check if I have ground beef in the fridge, show the instructions on the screen, and set a timer for 1 hour” (recipe search + fridge check + TV display + timer setting). These cases demand extensive reasoning, context understanding, and multi-device orchestration.

Each complexity group contains 20 test cases:

Intents Complexity	Number of Test Cases
Simple (1-2)	20
Moderate (3)	20
Complex (4+)	20
Total	60

Table 4.1: Benchmark distribution by intent complexity

Device Complexity

We further categorize the examples designed based on the number of intents according to the number of devices involved, exploring the system from another dimension.

This two-dimensional organization enables independent analysis of how input complexity and device coordination affect system performance.

Device Complexity	Number of Test Cases
1 device	10
2 devices	19
3 devices	6
4 devices	16
5+ devices	9
Total	60

Table 4.2: Benchmark distribution by device coordination complexity

4.3 Dataset Overview

This section provides an overview of the benchmark dataset. Table 4.3 summarizes device statistics across the three intent complexity groups. Figure 4.7 shows the percentage of each device within each test group.

Group	Intent Count	Avg Devices	Device Range
Simple	1-2	2.05	1-5
Moderate	3	3.20	2-5
Complex	4+	4.05	2-7
Total	1-8	3.10	1-7

Table 4.3: Average and Range of Device Count Across Intent Complexity Groups

As shown in Table 4.3, the average number of devices increases with intent complexity: simple cases average 2.05 devices, moderate cases average 3.20 devices, and complex cases average 4.05 devices. As expected, more complex intents require coordination of more devices. The device range also expands from 1-5 in simple cases to 2-7 in complex cases, reflecting increasing coordination requirements.

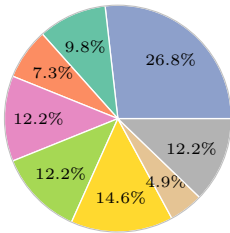
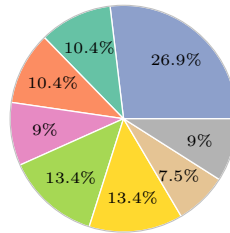
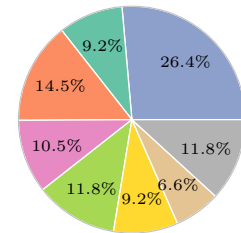
**Figure 4.4:** 1-2 intents**Figure 4.5:** 3 intents**Figure 4.6:** 4+ intents**Figure 4.7:** Device distribution across intent complexity groups

Figure 4.7 shows device distribution across complexity groups. Search Engine is the most frequently invoked device (26-27% across all groups), as smart home interactions

frequently require information retrieval—such as checking weather, searching recipes, or finding nearby services—either as standalone queries or in support of other tasks.

Simple cases show balanced distribution across devices. Moderate cases use Light and Audio more frequently (13.4% each) for scene-based requests. Complex cases show higher Calendar usage (14.5%) as these scenarios often involve scheduling and event planning. While some devices appear more than others, the overall distribution is fairly even. All eight devices are represented in each group.

4.4 Evaluation Criteria

To assess system performance in task planning and device collaboration, each test case is evaluated along three dimensions: Task Generation Quality, Collaboration Success, and Overall Completion.

Task Generation Quality

This dimension evaluates whether the Task Planner correctly decomposes user input into device-level tasks.

Correct:

All of the following conditions must be met:

- All user intents identified and addressed
- Appropriate devices selected based on their capabilities
- Task descriptions are clear, specific, and actionable
- Device capabilities properly matched to user requirements

Partial:

Any of the following conditions results in a partial rating:

- Most user intents addressed, but some omissions
- Suboptimal device selection (e.g., missing a relevant device or including unnecessary devices)
- Task descriptions vague but still actionable

Incorrect:

Any of the following conditions results in an incorrect rating:

- Major misinterpretation of user intent
- Wrong devices selected for the task
- Task descriptions unclear or unactionable

Collaboration Success

For test cases requiring coordination among multiple devices, this dimension evaluates whether agents correctly identify collaboration needs and execute collaboration effectively. Each collaboration instance is evaluated individually.

Successful:

A single collaboration is rated as successful when all of the following conditions are met:

- Agent correctly identifies the need for external information
- Obtains appropriate information from the correct source (via explicit request or shared state access)
- Collaboration response properly integrated

Failed:

A single collaboration is rated as failed when any of the following occurs:

- Agent fails to recognize the need for collaboration
- Requests information from the wrong agent
- Requests wrong or irrelevant information
- Fails to integrate collaboration response properly

Overall Completion

This dimension assesses end-to-end system performance in fulfilling user requests.

Complete:

A test case is rated as *Complete* when **all** of the following conditions are met:

- Task generation quality rated as “Correct”
- All required collaborations rated as “Successful”
- System produces coherent and complete final output

Partial:

A test case is rated as *Partial* when:

- The system has quality issues: Task generation rated as “Partial” OR some collaborations failed
- Despite these issues, the system produces output (though incomplete or with errors)

Failed:

A test case is rated as *Failed* when **any** of the following occurs:

- Task generation quality rated as “Incorrect”
- Collaborations failed such that the primary user intent cannot be fulfilled
- System unable to produce meaningful output

All benchmark scenarios were executed starting from the same initial system state. Responses were recorded and manually evaluated according to the criteria above, ensuring consistency and comparability across test cases.

5

Evaluation

This chapter evaluates the system using the 60-case benchmark from Chapter 4. Tests were conducted in three batches of 20 cases each, with system outputs manually recorded and evaluated according to the criteria defined in Section 4.4.

The evaluation examines overall system performance: performance by intent complexity, performance by device count, average response time, and common failure reasons.

5.1 Experimental Setup

All experiments were conducted using the following configuration:

Environment & Hardware: The system was implemented using Python 3.11 on a local workstation (MacBook Air) equipped with an Apple M4 chip and 16GB of unified memory.

Language Model: The system uses google/gemma-2-9b⁸, a lightweight open-source language model with 9 billion parameters, deployed via Ollama (version 0.13.5). The model is configured with zero temperature (temperature=0.0) to ensure deterministic and consistent outputs across all agents.

Framework: The multi-agent architecture is implemented using LangGraph (version 1.0.5), built upon the LangChain (version 1.2.0) framework. LangChain provides the interface for model interaction and prompt management, while LangGraph handles the stateful orchestration and coordination between the task planner and device agents.

5.2 Overall Performance

Performance by Intent Complexity

Table 5.1 shows a clear correlation between intent complexity and task completion rates. Simple cases (1-2 intents) achieved the highest success rate at 80%, with only 10% failures. As intents complexity increases, performance declines: moderate cases (3 intents) dropped to 55% success with 10% failures, while complex cases (4+ intents) achieved only 35% success with 20% failures.

⁸<https://huggingface.co/google/gemma-2-9b>

Group	Cases	Success	Partial	Failure
Simple (1-2)	20	16 (80%)	2 (10%)	2 (10%)
Moderate (3)	20	11 (55%)	7 (35%)	2 (10%)
Complex (4)	20	7 (35%)	9 (45%)	4 (20%)
Total	60	34 (56.7%)	18 (30%)	8 (13.3%)

Table 5.1: Performance by Intent Complexity

Importantly, partial success rates remained relatively stable across moderate and complex groups (30-40%), showing the system often partially processed tasks even when unable to complete them perfectly. However, the shift from 10% failures in simple cases to 20% in complex cases indicates that multi-intent scenarios introduce fundamental challenges beyond just increased coordination: they often involve implicit reasoning, conditional logic, or context understanding that the current system struggles to handle.

Overall, the system achieved 56.7% complete success across all 60 test cases. The 30% partial success rate indicates that many tasks were partially fulfilled, while 13.3% failed completely. Figure 5.2 visually represents the three complexity categories.

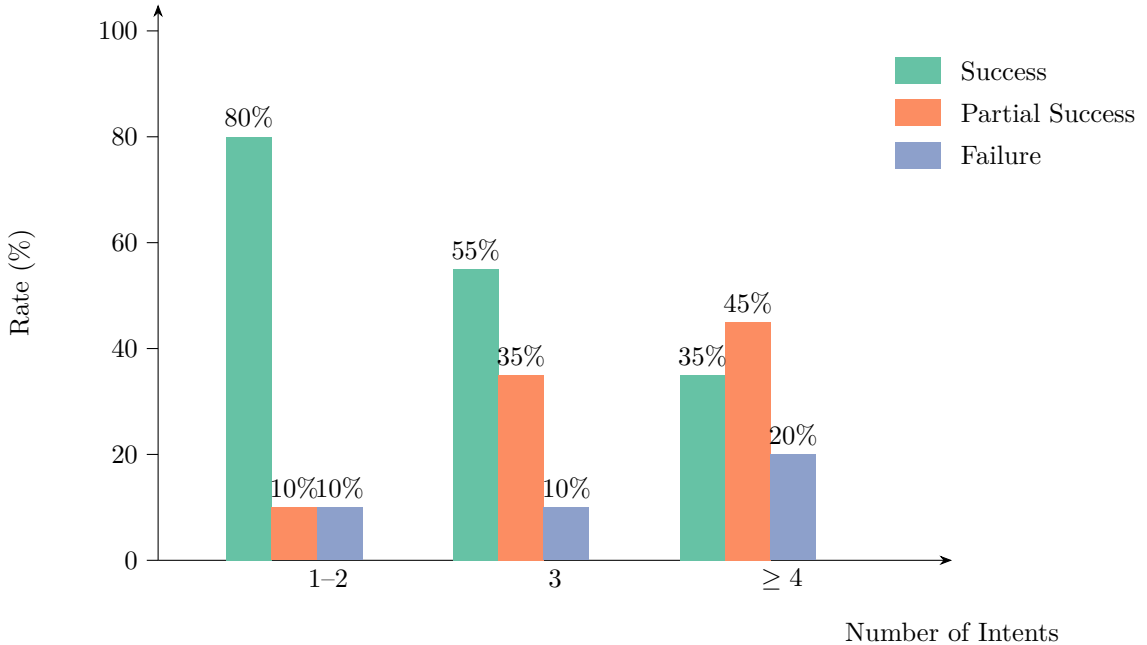
**Figure 5.2:** Performance by Number of Intents: Success, Partial Success, and Failure Rates.

Figure 5.2 shows an interesting thing in partial success rates. As intent complexity increases, partial success also increases: 10% for simple cases, 35% for moderate cases, and 45% for complex cases.

This indicates that multi-intent inputs may help the Task Planner better understand certain aspects of user requests. When users provide more intents, they also provide additional context about their situation and goals. This extra information helps the planner understand individual task components more clearly.

However, having too many intents also increases the failure rate. While additional context improves partial understanding, the coordination complexity required to complete all tasks ultimately leads to more failures.

Performance by Device Complexity

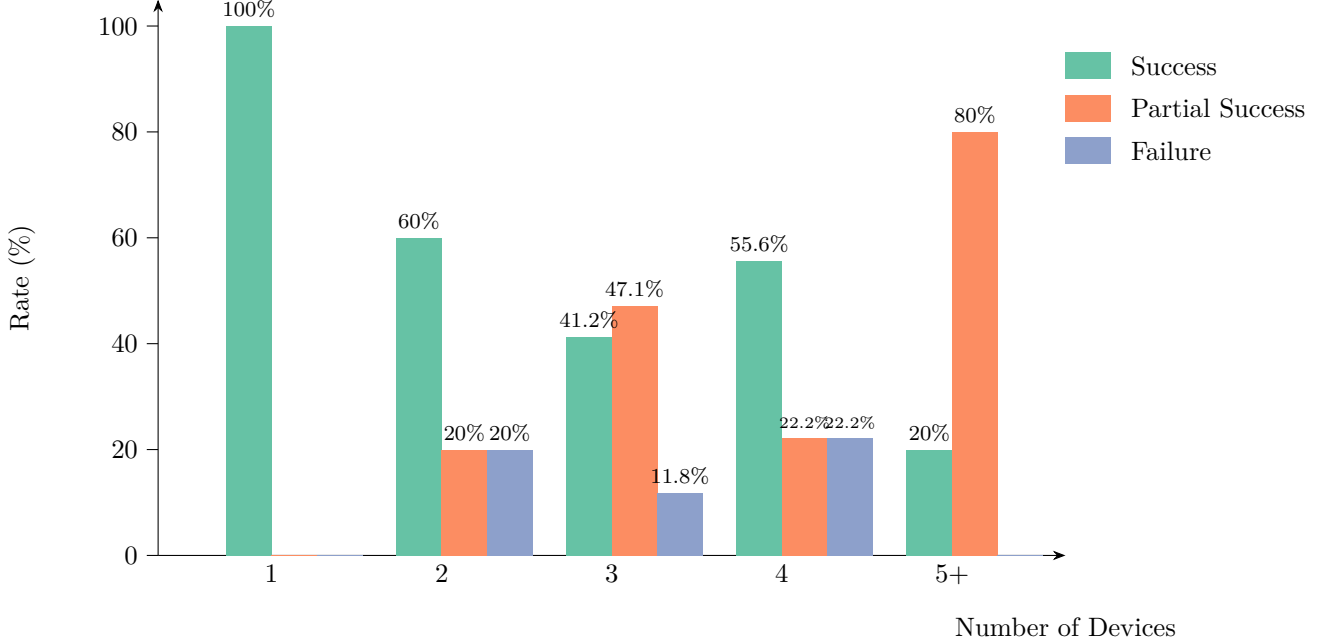


Figure 5.3: Performance by Number of Devices: Success, Partial Success, and Failure Rates.

Figure 5.3 shows how the number of devices affects system performance. Single-device tasks achieved a 100% success rate with no failures, demonstrating the system’s flawless capability for simple device commands. However, success rates decline as device count increases: 60% for two devices, 41.2% for three devices, and dropping to 20% for five or more devices.

The transition to multi-device scenarios reveals a clear trend towards partial completion. For scenarios involving five or more devices, the partial success rate peaks at 80%, while the success rate stands at only 20%. This suggests that high-device-count scenarios push the system into a “partial success zone” where it usually manages to invoke some devices correctly, but coordinating the entire chain of distinct devices becomes increasingly difficult.

Device count appears to have a stronger impact on performance than intent complexity. Increasing intent complexity (from Simple to Complex) reduced success by 45 percentage points (80% to 35%), while increasing the number of devices (from 1 to 5+ devices) reduced success by 80 percentage points (100% to 20%). This indicates that the architectural challenge of coordinating multiple distinct devices is significantly greater than handling multiple intents.

Response Time Analysis

Group	Avg Response Time (seconds)
Simple (1-2)	39.91
Moderate (3)	57.19
Complex (4)	78.18

Table 5.4: Average Response Time by Intent Complexity

Table 5.4 shows how response time changes with intents complexity. The average response time increases significantly with intent complexity. Simple tasks averaged 39.91s, moderate tasks rose to 57.19s (+43%), and complex tasks reached 78.18s (+96% compared to simple cases). The increase is due to several factors working together. Complex cases involve more devices (4.05 on average), and each device requires LLM processing. Also, complex cases often involve device collaboration—for example, Fridge asking Search Engine for recipes, or TV asking Calendar for schedules. Although each device collaborates at most once, several devices may need collaboration within a single task. Since devices execute one after another rather than at the same time, these processing steps add up, leading to much longer response times.

5.3 Performance Analysis by Case Examples

This section presents representative examples illustrating different system outcomes: fully successful execution, partially successful case, and failed case. Each example demonstrates the system’s behavior under varying levels of task complexity and inter-device dependencies.

Successful Case 1

The weather’s awful today. Check the outdoor temperature for me, then set the indoor temperature a bit warmer than outside

Task Planner generate Task Queue:

1. Search engine: check the outdoor temperature.
2. Thermostat: set the indoor temperature a bit warmer than outside.

Execution Flow:

- Search engine Agent: Hamburg outdoor temperature: 15°C, sunny.
- Thermostat Agent: Setting indoor temperature to 18°C, warmer than the current outdoor temperature of 15°C.

Successful Case 2

I want to make 'Chicken Alfredo', Check my fridge and tell me what I’m missing

Task Planner generate Task Queue:

1. Fridge Agent requests collaboration from Search Engine Agent: What ingredients are needed for Chicken Alfredo?
2. Search Engine Agent responds to Fridge Agent: Chicken breasts, fettuccine pasta, heavy cream, butter, Parmesan cheese, garlic, salt, pepper.
3. Fridge Agent: You have Chicken breasts, fettuccine pasta, butter, Parmesan cheese, garlic, salt, and pepper in your fridge. You are missing heavy cream to make Chicken Alfredo.

Execution Flow:

- Fridge Agent requests collaboration from Search Engine Agent: What ingredients are needed for Chicken Alfredo?
- Search Engine Agent responds to Fridge Agent: Chicken breasts, fettuccine pasta, heavy cream, butter, Parmesan cheese, garlic, salt, pepper.
- Fridge Agent: You have Chicken breasts, fettuccine pasta, butter, Parmesan cheese, garlic, salt, and pepper in your fridge. You are missing heavy cream to make Chicken Alfredo.

Successful Case 3

Can you show me my schedule on the tv screen?

Task Planner generate Task Queue:

1. TV display: show my schedule on the tv screen.

Execution Flow:

- TV display Agent requests collaboration from Calendar Agent: get today’s schedule and appointments
- Calendar Agent responds to TV display Agent: Today’s Schedule: 10:00 AM - 11:00 AM: Dentist Appointment (Dr. Smith, 123 Main Street), 2:00 PM - 3:00 PM: Meeting with John Doe (Conference Room A), 6:00 PM - 7:00 PM: Dinner with Family (Home).
- TV display Agent: Today’s Schedule: 10:00 AM - 11:00 AM: Dentist Appointment (Dr. Smith, 123 Main Street), 2:00 PM - 3:00 PM: Meeting with John Doe (Conference Room A), 6:00 PM - 7:00 PM: Dinner with Family (Home).

Analysis of Successful Cases

These three cases highlight what the system does well.

In Case 1, the task queue made sense: check outdoor temperature first, then adjust indoor temperature. More importantly, the Thermostat actually used the Search Engine’s result (15°C) when deciding to set indoor temperature to 18°C. This shows the agents can properly share and use information.

Case 2 worked well because the Fridge was the right agent to handle this request. When the Fridge realized it needed to know what ingredients “Chicken Alfredo” requires, it asked the Search Engine for help. The Search Engine provided the ingredient list, and the Fridge used that to figure out what was missing. The whole collaboration flow was logical.

Case 3 is simpler but still demonstrates good planning. Giving the task to TV Display made sense, and when TV Display needed schedule data, it asked Calendar. The collaboration worked as expected.

All three cases succeeded because tasks went to the right devices, and when agents needed extra information, they knew which other agent to ask.

Partially Successful Case 1

Play anything for 1 hour

Task Planner generate Task Queue:

1. Audio system Agent: play anything for 1 hour.

Execution Results:

- Audio system Agent requests collaboration from Clock Agent: set 1 hour timer for playing music.
- Clock responds to Audio system Agent: Timer set for 1 hour.
- Audio system Agent: Playing music for 1 hour.

Partially Successful Case 2

I'm feeling sick today and need to rest. The room should be comfortable, but there's no need to adjust the temperature

Task Planner generate Task Queue:

1. Lighting: make the room comfortable, as requested by the user
2. Audio system: play relaxing music for a comfortable atmosphere

Execution Results:

- Lighting Agent: Setting soft white light at 60% brightness for a comfortable and relaxing atmosphere.
- Audio system Agent: Playing relaxing music for a comfortable atmosphere.

Partially Successful Case 3

I'm training for a marathon and need structure. Wake me at 5:30am, show my training schedule, play motivating music, set a timer for 5 min, and remind me about proper meal times

Task Planner generate Task Queue:

1. Clock Agent: wake user at 5:30.
2. Calendar Agent: show training schedule.
3. Audio system Agent: play motivating music.
4. Clock Agent: set a timer for 5 minutes.
5. Calendar Agent: remind user about proper meal times

Execution Results:

- Clock Agent: Alarm set for 5:30.
- Calendar -> TV_display Agent: Display your training schedule.
- TV_display Agent responds: Training Schedule: Monday: 9:00 AM - 12:00 PM: Natural Language Processing Tuesday: 1:00 PM - 4:00 PM: Computer Vision Wednesday: 10:00 AM - 1:00 PM: Dialogue Systems Thursday: 2:00 PM - 5:00 PM: Machine Learning Fundamentals Friday: 9:00 AM - 12:00 PM: Smart Home Integration.
- Audio -> Search Agent: Recommend motivating music.
- Search Agent responds: Uptown Funk by Mark Ronson ft. Bruno Mars, Eye of the Tiger by Survivor, Don't Stop Believin' by Journey, We Will Rock You by Queen, Stronger (What Doesn't Kill You) by Kelly Clarkson.
- Audio system Agent: Playing motivating music: Uptown Funk by Mark Ronson ft. Bruno Mars.
- Clock Agent: Timer set for 5 minutes.
- Calendar Agent: It's important to eat meals at regular intervals throughout the day. Try to have breakfast within an hour of waking up, lunch around midday, and dinner in the early evening. Remember to stay hydrated by drinking plenty of water.

Analysis of Partially Successful Cases

Case 1 shows a missing collaboration step. The primary cause is that the task planner agent doesn't assign tasks properly. Furthermore, since a device agent can only collaborate once, this is likely the main reason audio doesn't initiate collaboration with the search engine.

Case 2 shows incomplete collaboration. The Audio agent said it would play "relaxing music" but didn't actually specify what to play. Since Audio cannot play non-specific content, it should have first asked Search Engine for relaxing music recommendations, then selected specific tracks. The task assignment was correct, but the missing collaboration step meant the task wasn't fully successful.

Case 3 shows that because the tasks in the task queue generated by the task planner lack contextual descriptions, the training schedule gives a course schedule completely unrelated to the marathon. Tasks in the task queue may require an additional verification layer.

These cases show two related problems: agents sometimes miss necessary collaboration while other times perform unnecessary collaboration. This reflects a limitation of LLM-based agents: unstable performance across similar scenarios. This inconsistency may result from subtle variations in user expressions or the inherent uncertainty in LLM outputs.

Failed Case 1

I'm working on an important deadline tonight. I need optimal lighting and temperature, some focus music, then check what time is my project deadline

Task Planner generate Task Queue:

1. Lighting: adjust lights for optimal lighting.
2. Thermostat: set temperature for optimal comfort.
3. Audio: play focus music.
4. Clock: check the time of my project deadline.

During the task queue phase, it became obvious that when asked about time, the LLM sometimes simply passed the query to the clock—which is completely wrong.

Failed Case 2

I'm feeling lonely and bored. I want some entertainment and want a cozy environment, but please keep it quiet

Task Planner generate Task Queue:

1. Audio: play music for entertainment, keep the volume low and creating a cozy atmosphere.
2. Lighting: dim the lights to create a cozy environment.

Execution Flow:

- Audio system Agent: Playing some mellow tunes at a low volume to create a cozy atmosphere.
- Lighting Agent: Dimming the lights to 50% brightness and setting a warm white color temperature for a cozy ambiance.

Failed Case 3

Help me find an Italian pasta recipe and see what ingredients I'm missing

Task Planner generate Task Queue:

1. Search Engine: Find an Italian pasta recipe and see what ingredients I'm missing

Execution Flow:

- Search Engine Agent: Found 3 Italian pasta recipes: 1) Spaghetti Carbonara, 2) Fettuccine Alfredo, 3) Lasagna. Each takes 30-45 minutes.

Analysis of Failed Cases

The Task Planner tends to ignore implicit intents in multi-intent requests. Context-setting phrases like “good night’s sleep” and “relax” were treated as irrelevant, causing the planner to generate incomplete tasks. This intent-ignoring behavior also appears at the device level: in Case 3, the Search Engine agent addressed only part of the request. These issues indicate that prompt engineering for both Task Planner and device agents requires further refinement to improve recognition of implicit intents and multi-part tasks.

5.4 Discussion

This section discusses the findings from the evaluation, examining both what the system does well and where improvements are needed.

System Strengths

The evaluation demonstrates that LLM-based multi-agent systems are viable for smart home systems, with several notable strengths.

Strong Performance on Direct Commands: The system excels at handling explicit, well-structured requests. Single-device tasks achieved 100% success with zero complete failures, demonstrating reliable execution when user intents directly map to device capabilities. Even for simple multi-intent cases (1-2 intents), the system maintained 80% success, showing effective task decomposition and collaboration. This indicates that for a substantial portion of daily smart home interactions—straightforward commands like “Turn on the lights” or “Display my schedule on the screen”—the system performs reliably.

Device Collaboration: The device collaboration mechanism performs well in well-defined scenarios. Agents autonomously recognize when information from other agents is required and successfully execute collaborative tasks. Examples include the refrigerator requesting recipe data from a search engine or the TV display retrieving schedules from a calendar.

Effective Architectural Design: The hybrid architecture combining centralized planning with decentralized collaboration proves practical. The Task Planner successfully decomposes complex requests into device-level tasks without needing to predict all collaboration details, while device agents autonomously handle coordination when needed. This separation of concerns allows the system to scale: new devices can be added as independent agents without modifying the central planner.

Natural Language as user input: Unlike traditional rule-based systems, users can express requests using natural language. This system handles multiple phrasing variations and can process implicit requests within simple inputs, offering a more conversational interaction model compared to predefined automated rules.

Challenges

While the system shows promise, several challenges emerged that require attention.

Implicit Intent Ignored: Despite multiple refinements to the Task Planner’s prompts, LLM performance on implicit intent recognition remains inconsistent. The observations show that when user input is a single sentence containing ambiguous intent, the LLM can usually identify and analyze it. However, when input consists of multiple clauses, the LLM occasionally overlooks implicit intents. This suggests that the current Intent Analyzer and Task Planner may not be sufficient to ensure stable intent understanding. The system might need additional agent components that guide the LLM through multi-step reasoning to handle implicit and explicit intents more evenly. At the same time, it is also necessary to avoid deep reasoning for every ambiguous statement to prevent efficiency issues caused by overanalysis.

Scaling to Multi-Device Scenarios: Performance declines as device count increases, from 100% success for single devices to 20% for five or more devices. The drop at three devices (from 60% to 41.2% success) suggests a coordination complexity threshold. However, the high partial success rates indicate the core mechanisms work. The main challenge is achieving complete coordination across all devices.

Behavioral Consistency: The evaluation showed the Fridge agent behaves inconsistently across similar recipe requests. For “cookies”, it acted as if it knew recipes and responded directly. For “Chicken Alfredo”, it recognized it needed recipe information and requested help from the Search Engine. This inconsistency persists despite the Fridge agent’s prompt explicitly stating it lacks recipe capabilities. This reveals the LLM’s unclear understanding of the fridge’s functional boundaries, possibly seeing it as both a storage device and recipe provider. Future work needs clearer function definitions and prompt design to better define each agent’s capabilities and reduce the LLM’s uncertainty about what devices can do.

Response Time: Complex tasks average 78 seconds, primarily due to sequential execution and multiple LLM calls. While this is manageable for non-urgent requests, faster response times would improve user experience.

Summary

The evaluation shows that LLM-based multi-agent systems can work for smart homes, especially for straightforward tasks. The system succeeded completely on 56.7% of test cases and partially on another 35%, meaning it handled most requests at least partly correctly. For simple, direct commands, performance was strong: 100% success for single-device tasks and 80% for simple intent cases.

The main problems are with complex scenarios: implicit intents that aren’t clearly stated, coordinating many devices at once, and LLM inconsistency. But these seem fixable with better prompts and additional mechanisms rather than requiring a complete redesign.

The bigger meaning is that this work proves the approach can work. Using open-source LLMs with LangGraph is practical and cost-effective. The hybrid architecture of central planning and decentralized dynamic device collaboration functions. The challenge now is making it work more reliably and consistently.

6

Conclusion

6.1 Summary

The research addressed two core challenges: how to design a central planner that effectively decomposes complex natural language requests without predicting collaboration details, and whether decentralized, peer-to-peer dynamic collaboration is sufficient for LLM-based device agents to autonomously coordinate.

Addressing the cost limitations identified in SAGE [16], we implemented all agents using locally-deployed open-source LLMs rather than expensive proprietary models. We designed a multi-agent architecture featuring an Intent Analyzer agent, a Task Planner agent, and eight autonomous device agents (Calendar, Clock, Search Engine, Light, Thermostat, Audio System, TV Display, Fridge). The system was implemented using LangGraph, which provides state sharing for information exchange, command-based routing where agents control execution flow through goto commands, and message passing for device collaboration. This demonstrates that LangGraph’s architecture can effectively support the hybrid approach of centralized planning with decentralized coordination.

To evaluate the system, we created a benchmark of 60 test cases organized by intent complexity. Evaluation revealed that the system achieved 56.7% complete success overall, with strong performance on explicit requests (100% for single-device tasks, 80% for simple cases) but struggled with implicit intent recognition and multi-device coordination (35% success for complex cases, 20% for 5+ devices). Response times ranged from 44 seconds for simple tasks to 141 seconds for complex tasks.

These results demonstrate that while LLM-based multi-agent approaches show promise for smart home systems, improvements in implicit intent understanding, behavioral consistency, multi-turn interactions and coordination efficiency are needed for practical deployment.

6.2 Contributions

This thesis makes four primary contributions to the field of LLM-based multi-agent systems for smart home systems:

Hybrid Multi-Agent Architecture: We designed a system combining centralized

task planning with decentralized device dynamic collaboration. The Task Planner decomposes user requests into high-level tasks without predicting collaboration, while device agents autonomously determine collaboration needs. This demonstrates effective integration of both coordination approaches using LLMs.

Dynamic Collaboration Mechanism: We implemented autonomous agent collaboration where devices use LLM-based reasoning to identify information needs and request help from peers, rather than following pre-defined rules. This enables flexible, context-aware coordination without centralized orchestration.

Functional Proof-of-Concept: We developed a working system with 8 device agents (Calendar, Clock, Search Engine, Light, Thermostat, Audio, TV Display, Fridge) implemented using LangGraph and locally deployed open-source LLMs. The implementation leverages LangGraph’s state-sharing mechanisms, command-based routing, and message-passing capabilities, demonstrating the practical feasibility of the proposed architecture and showing that cost-effective smart home systems can be built without relying on expensive proprietary LLMs.

Domain-Specific Benchmark: We created and applied a benchmark of 60 test cases organized by intent complexity (simple, moderate, complex), with 20 cases in each group. This benchmark addresses the lack of domain-specific evaluation frameworks for smart home multi-agent systems, as existing benchmarks focus on general-purpose tasks rather than smart home scenarios. The evaluation provided insights into system performance across different complexity levels and device counts, revealing both the promise of LLM-based approaches (90% success for single-device tasks) and their limitations (struggles with implicit intents and multi-device coordination).

6.3 Future Work

There are several directions to improve the system’s robustness and usability in future iterations.

First, to enhance natural language understanding, each agent could be equipped with its own NLU (Natural Language Understanding) module. This distributed approach would allow agents to handle device-specific semantics more accurately compared to a single global model. Second, the system should support multi-turn dialogue. Currently, the system may fail when requests are ambiguous. Future work involves adding a feedback mechanism where the agent asks clarifying questions to confirm user intent, preventing errors caused by wrong assumptions. Third, to ensure reliability, stricter output constraints and validation layers should be implemented. Agents also need a verification step to check if a generated task falls within their actual capabilities (scope) before execution. Finally, evaluation and personalization need to be expanded. To increase the test suite to 100+ cases, benchmark against different open-source models (e.g., Llama, Mistral), and test with real physical devices. Additionally, a memory module could be added to learn user habits—such as preferred temperatures, so that the system can proactively apply settings without repetitive instructions.

Bibliography

- [1] Acharya, D. B., Kuppan, K., and Divya, B. Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey. In: *IEEE Access*, 2025.
- [2] Agashe, S., Fan, Y., Reyna, A., and Wang, X. E. Llm-coordination: evaluating and analyzing multi-agent coordination abilities in large language models. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. 2025, pp. 8038–8057.
- [3] Agashe, S., Fan, Y., and Wang, X. E. Evaluating multi-agent coordination abilities in large language models. In: 2023.
- [4] Alam, M. R., Reaz, M. B. I., and Ali, M. A. M. A review of smart homes—Past, present, and future. In: *IEEE transactions on systems, man, and cybernetics, part C (applications and reviews)* 42(6):1190–1203, 2012.
- [5] Aldrich, F. K. Smart homes: past, present and future. In: *Inside the smart home*. Springer, 2003, pp. 17–39.
- [6] Ali, S. and Yusuf, Z. Mapping the smart-home market. In: *Boston: The Boston Consulting Group*, 2018.
- [7] Bhat, A., Mondal, A., and Tripathy, A. LLM Agents for Internet of Things (IoT) Applications. In: 2025.
- [8] Dorri, A., Kanhere, S. S., and Jurdak, R. Multi-agent systems: A survey. In: *Ieee Access* 6:28573–28593, 2018.
- [9] Halba, K., Griffor, E., Lbath, A., and Dahbura, A. IoT capabilities composition and decomposition: A systematic review. In: *IEEE Access* 11:29959–30007, 2023.
- [10] King, E., Yu, H., Lee, S., and Julien, C. Sasha: creative goal-oriented reasoning in smart homes with large language models. In: *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8(1):1–38, 2024.
- [11] Madakam, S. Internet of things: smart things. In: *International journal of future computer and communication* 4(4):250, 2015.
- [12] Mohammadi, M., Li, Y., Lo, J., and Yip, W. Evaluation and benchmarking of llm agents: A survey. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 2025, pp. 6129–6139.
- [13] Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., and Mian, A. A comprehensive overview of large language models. In: *ACM Transactions on Intelligent Systems and Technology* 16(5):1–72, 2025.
- [14] Pelluru, K. LangChain & LangGraph in Production: Architectures for Multi-Agent LLM Systems. In: *Journal of Data and Digital Innovation (JDDI)* 2(3):1–9, 2025.
- [15] Ricquebourg, V., Menga, D., Durand, D., Marhic, B., Delahoche, L., and Loge, C. The smart home concept: our immediate future. In: *2006 1st IEEE international conference on e-learning in industrial electronics*. IEEE. 2006, pp. 23–28.

Bibliography

- [16] Rivkin, D., Hogan, F., Feriani, A., Konar, A., Sigal, A., Liu, S., and Dudek, G. Sage: Smart home agent with grounded execution. In: *arXiv preprint arXiv:2311.00772*, 2023.
- [17] Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O’Sullivan, B., and Nguyen, H. D. Multi-agent collaboration mechanisms: A survey of llms. In: *arXiv preprint arXiv:2501.06322*, 2025.
- [18] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., et al. A survey on large language model based autonomous agents. In: *Frontiers of Computer Science* 18(6):186345, 2024.
- [19] Wilsker, B. *A Study of Multi-Agent Collaboration Theories*. Tech. rep. 1996.
- [20] Yang, H., Lee, W., and Lee, H. IoT smart home adoption: the importance of proper level automation. In: *Journal of Sensors* 2018(1):6464036, 2018.
- [21] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In: *The eleventh international conference on learning representations*. 2022.
- [22] Zhu, K., Du, H., Hong, Z., Yang, X., Guo, S., Wang, D. Z., Wang, Z., Qian, C., Tang, R., Ji, H., et al. Multiagentbench: Evaluating the collaboration and competition of llm agents. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025, pp. 8580–8622.



Source Code Repository

The complete source code for this thesis is publicly available on GitHub:

- **System Implementation:**
<https://github.com/katzezweii/smart-home-llm-multi-agent>
- **Benchmark Dataset:**
<https://github.com/katzezweii/smart-home-llm-multi-agent/tree/main/benchmark>

The repository includes:

- Multi-agent system implementation
- All test cases and evaluation scripts
- Configuration files and documentation

B

Test Cases and Results

B.1 Simple Test Cases

ID	User Input	Result	Time (s)
S01	Help me find an Italian pasta recipe and see what ingredients I'm missing	Failure	36.83
S02	What's in my fridge, and what can I make for dinner?	Success	39.24
S03	I think I have some chicken breast in the fridge. What low-calorie meal can I make with it?	Success	35.56
S04	I want to bake cookies. Do I have all the ingredients I need?	Failure	33.14
S05	I'm going to study	Success	35.26
S06	: play anything for 1 hour	Partial Success	37.22
S07	Set a 20 minutes timer for my laundry	Success	35.25
S08	Are there any barbershops nearby?	Success	36.57
S09	Increase the bedroom temperature	Success	32.08
S10	Add a 'Team Meeting' tomorrow at 10am	Success	37.62
S11	I want to watch Stranger Things Season 3	Success	34.59
S12	Play some classical music, but not too loud	Success	46.92
S13	Set the lights for reading	Success	33.36
S14	What time does my next schedule begin	Success	43.95
S15	It's so hot today, I'm going to cook lunch	Success	42.94
S16	Can you dim the bedroom lights and turn off the living room lights?	Success	42.18
S17	Can you show me my schedule on the tv screen?	Success	55.14
S18	Is there any food in the house that we can eat right now?	Success	38.36
S19	I need to sleep well and wake up on time	Partial Success	45.01
S20	I'm having friends over for dinner soon, make the living room cozy	Success	56.89

Table B.1: Simple Test Cases Complete Results

B.2 Moderate Test Cases

ID	User Input	Result	Time (s)
M01	I've just finished today's work, now I need to relax and check my tomorrow's schedule	Partial Success	31.33
M02	I need to study for 45 minutes, brighten the light and play some focus music	Success	50.15
M03	Find me a good chicken recipe based on what I have in the fridge, and show the recipe instructions on the TV	Partial Success	67.61
M04	I want to make 'Chicken Alfredo', Check my fridge and tell me what I'm missing	Success	50.11
M05	Check the weather for tomorrow, and set a reminder to bring an umbrella if it's going to rain	Success	53.51
M06	I'm going to cook, see what food is about to expire, and play some upbeat music	Partial Success	80.95
M07	Set a wake-up alarm for tomorrow 7 AM, remind me to 'Call Mom' tonight, and also start a 10-minute timer for the coffee	Success	58.00
M08	I just got home from work and feel exhausted. The room feels too cold and dark	Success	46.2
M09	The house feels stuffy and uncomfortable. I want to adjust the environment and maybe play some music	Success	53.98
M10	I'm feeling lonely and bored. I want some entertainment and want a cozy environment, but please keep it quiet	Failure	49.13
M11	I'm feeling sick today and need to rest. The room should be comfortable, but there's no need to adjust the temperature	Partial Success	47.95
M12	Display the refrigerator's inventory on the screen, then show tomorrow's schedule	Partial Success	81.45
M13	turn on the light in the living room, show me some TV shows and set a one-hour countdown timer for watching TV	Success	66.34
M14	I'm feeling overwhelmed with my schedule. Show me what's coming up and help me prepare	Failure	41.86
M15	Play some jazz music, watch yoga practice videos, and then turn down the temperature a bit	Partial Success	73.57
M16	I'd like to rest for a bit. Please wake me up before the next appointment	Success	46.38
M17	I'm trying to cook dinner, but I don't know what's in the fridge. I need some quick recipe ideas	Success	67.76
M18	I'm craving pasta. Check if we have enough ingredients in the fridge. If anything's missing, please list it.	Partial Success	41.80
M19	show a movie, turn off the light and adjust the living room ambiance	Success	68.74
M20	Please make the lights brighter, play the 'Focus' playlist, and what do I have scheduled an hour from now?	Success	67.01

Table B.2: Moderate Test Cases Complete Results

B.3 Complex Test Cases

B Test Cases and Results

ID	User Input	Result	Time (s)
C01	I want to cook the chicken from the fridge. Find a recipe, display the details on the TV, and set a 15-minute timer for the first step	Success	101.63
C02	I'm planning a dinner party for this Friday night. First, check my calendar to see if I'm free. If I am, check the fridge for what I have, and then search for suitable dinner party recipes	Partial Success	53.9
C03	I'm getting ready for bed. Play some relaxing 'sleep' sounds, dim all the lights, and set an alarm for 7 AM tomorrow	Partial Success	51.6
C04	Please turn on the bedroom lights, tell me my today's first meeting details, and what's the weather like? If it rains, remind me to bring an umbrella	Failure	72.09
C05	Find me a lasagna recipe, check if I have ground beef in the fridge, show the instructions on the screen, and set a timer for 1 hour	Success	99.11
C06	I'm recovering from surgery and need to rest. Make the home comfortable, remind me about medication every 4 hours, suggest easy meals I can prepare, show me something entertaining, and track my doctor's appointment tomorrow	Partial Success	120.80
C07	I'm training for a marathon and need structure. Wake me at 5:30am, show my training schedule, play motivating music, set a timer for 5 min, and remind me about proper meal times	Partial Success	120.10
C08	The weather forecast says it's going to be cold tonight. I want to stay warm and cozy to watch a movie, and I should also check what I can make for dinner	Partial Success	53.23
C09	I'm feeling really stressed and can't concentrate. I need a better working environment, some background music, and I should also check when my next deadline is	Success	79.01
C10	My friends are staying over for the weekend. I need to plan meals for those days and check if I have enough food in the fridge, then list the ingredients for some simple and quick meals	Partial Success	54.28
C11	I just got home from work and I'm feeling exhausted. The weather forecast says it's going to be cold tonight. I want to create a cozy atmosphere for relaxation	Success	49.95
C12	I'm working on an important deadline tonight. I need optimal lighting and temperature, some focus music, then check what time is my project deadline	Failure	90.91
C13	I'm babysitting my nieces tomorrow. Check what kid-friendly food I have, suggest meal ideas, help me plan activities, show appropriate entertainment, and remind me of their nap time at 2pm	Partial Success	109.52
C14	Check my schedule for tomorrow and see if I have any free time. If I do, pick a hair salon for me and add this to my schedule	Failure	72.82
C15	I'm hosting Thanksgiving dinner and getting overwhelmed. Show me recipes based on what I have, help me plan the cooking timeline, set the festival ambiance, and play appropriate music	Success	117.76
C16	Check the fridge for milk. If there's milk, add an event to tomorrow's schedule: 'Go to the supermarket at 5 PM to buy cereal.' If there's no milk, add an event: 'Go to the supermarket at 5 PM to buy milk and cereal'	Failure	53.01
C17	I want to watch tv show for an hour, and set the living room to 24 degrees for two hours, then show me tomorrow's schedule before I go to bed	Partial Success	100.10
C18	Remind me in ten minutes to close the refrigerator door. If the light in kitchen is still on, then turn it off	Success	49.98
C19	The weather's awful today. Check the outdoor temperature for me, then set the indoor temperature a bit warmer than outside	Success	50.72
C20	I'm trying to establish a bedtime routine. I want the lights to dim gradually starting at 10pm. Keep the temperature as it is now. Also play some quiet sounds to help me sleep	Partial Success	63.00

Table B.3: Complex Test Cases Complete Results